

Arquiteturas de Deep Learning

REDES NEURAIS PROFUNDAS

Zenilton K. G. Patrocínio Jr

zenilton@pucminas.br

Aprendizagem Profunda → Aprendizagem de Representação

Aprendizagem de representação em camadas

Aprendizagem Profunda → Aprendizagem de Representação

Aprendizagem de representação em camadas



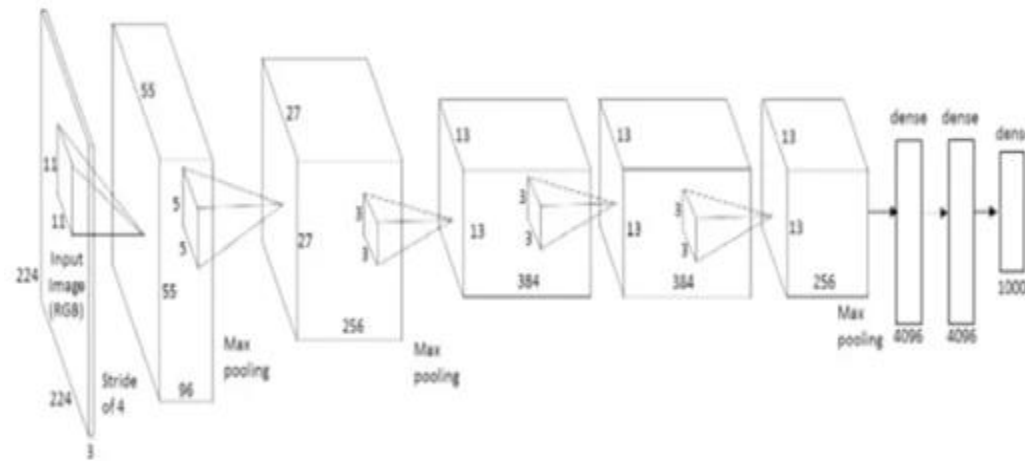
1,4 M imagens

Aprendizagem Profunda → Aprendizagem de Representação

Aprendizagem de representação em camadas



1,4 M imagens

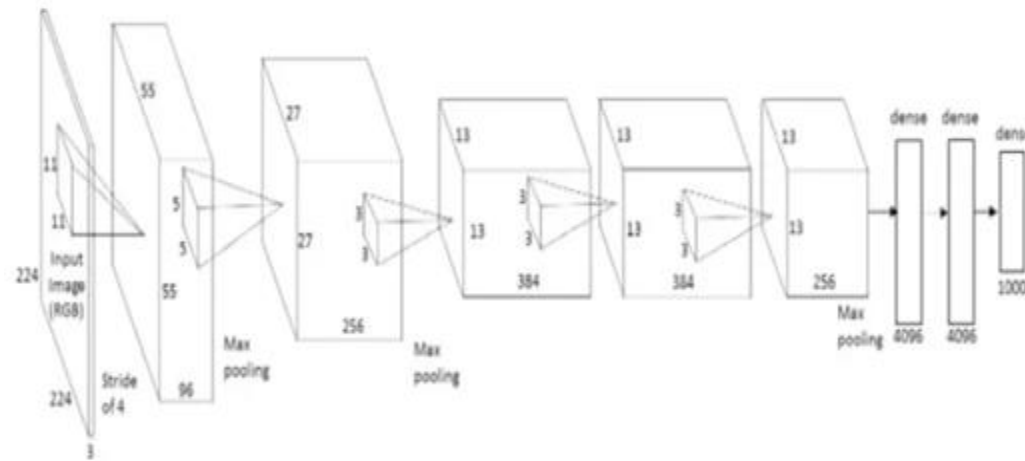


Aprendizagem Profunda → Aprendizagem de Representação

Aprendizagem de representação em camadas



1,4 M imagens



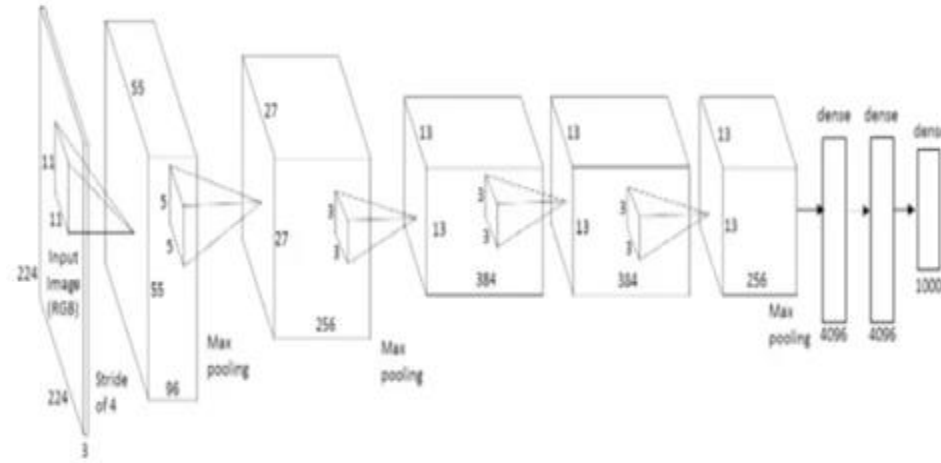
- 0: 'tench, Tinca tinca',
- 1: 'goldfish, Carassius auratus',
- 2: 'great white shark, white shark, man-eater, man-eating
- 3: 'tiger shark, Galeocerdo cuvieri',
- 4: 'hammerhead, hammerhead shark',
- 5: 'electric ray, crampfish, numbfish, torpedo',
- 6: 'stingray',
- 7: 'cock',
- 8: 'hen',
- 9: 'ostrich, Struthio camelus',
- 10: 'brambling, Fringilla montifringilla',
- 11: 'goldfinch, Carduelis carduelis',
- 12: 'house finch, linnet, Cardopacus mexicanus',
- 13: 'junco, snowbird',
- 14: 'indigo bunting, indigo finch, indigo bird, Passerina
- 15: 'robin, American robin, Turdus migratorius',
- 16: 'bulbul',
- 17: 'jay',
- 18: 'magpie',
- 19: 'chickadee',
- 20: 'water ouzel, dipper',
- 21: 'kite',
- 22: 'bald eagle, American eagle, Haliaeetus leucocephalus'
- 23: 'vulture',
- 24: 'great grey owl, great gray owl, Strix nebulosa',

⋮

1.000 classes

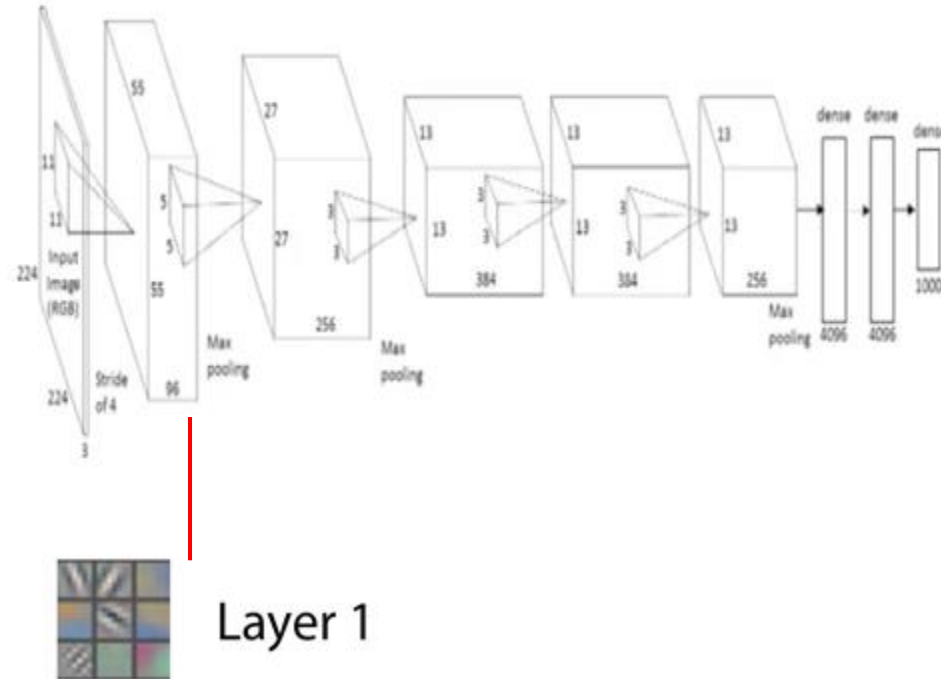
Redes Neurais Profundas

Após o treinamento...



Redes Neurais Profundas

Após o treinamento...

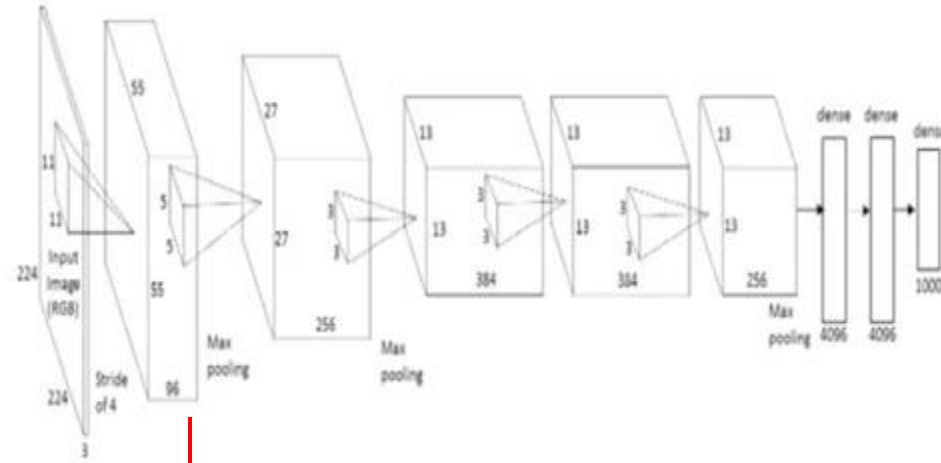


Resposta neural :

- cinza é zero (sem mudança)
- preto é negativo
- branco é positivo

Redes Neurais Profundas

Após o treinamento...



Resposta neural:

- cinza é zero (sem mudança)
- preto é negativo
- branco é positivo

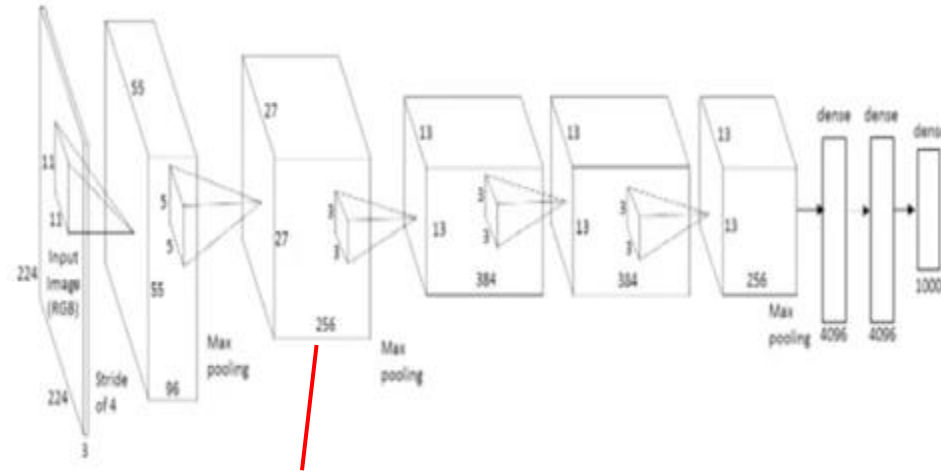


Layer 1

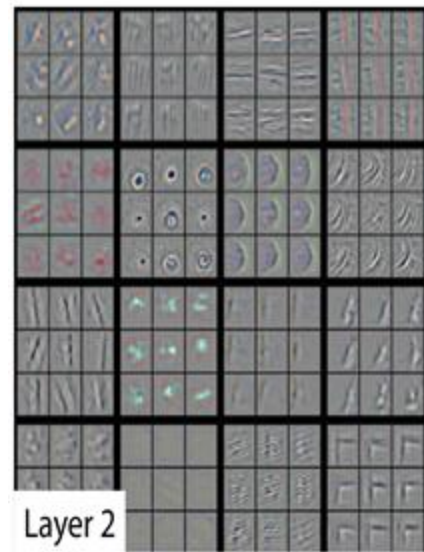


Regiões da imagem com respostas mais fortes

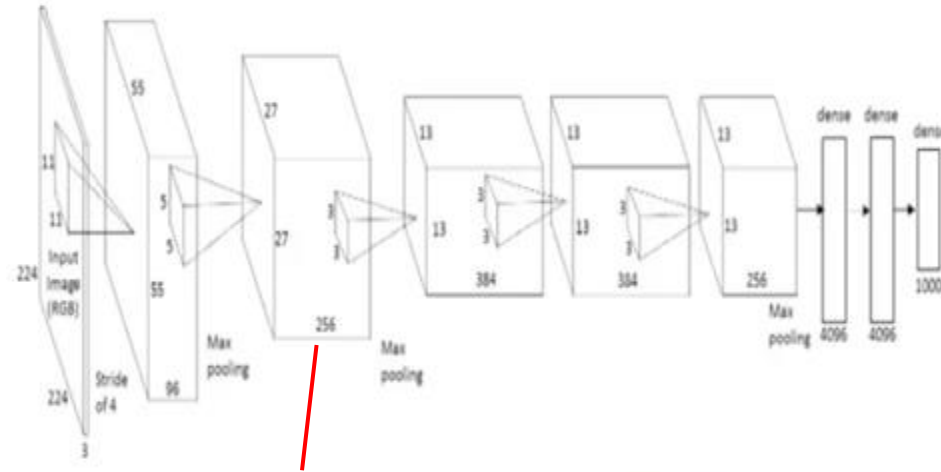
Redes Neurais Profundas



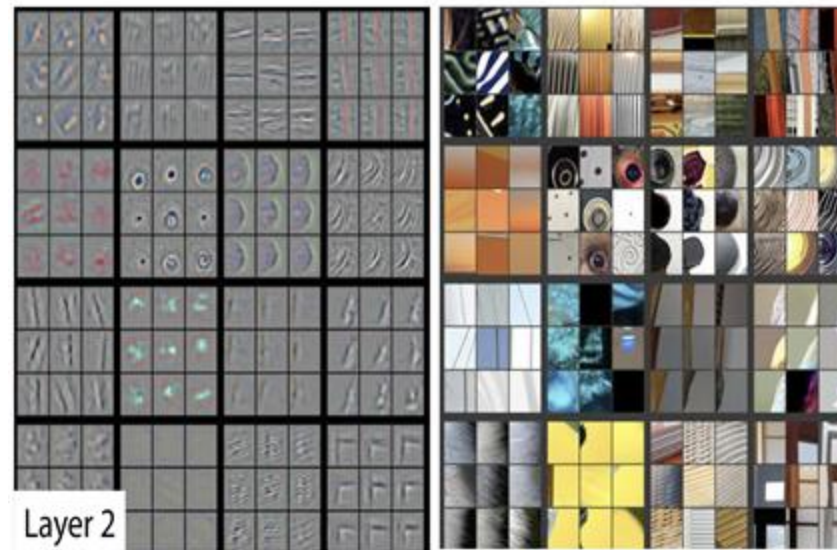
Resposta neural:



Redes Neurais Profundas

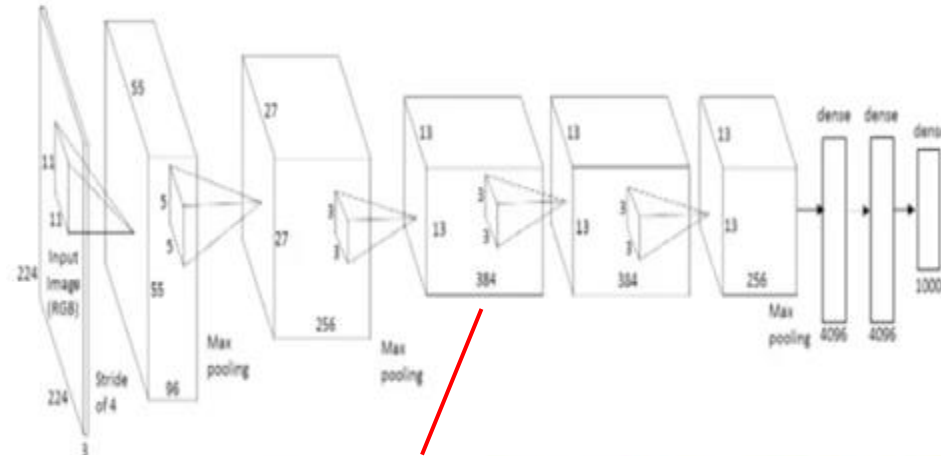


Resposta neural:

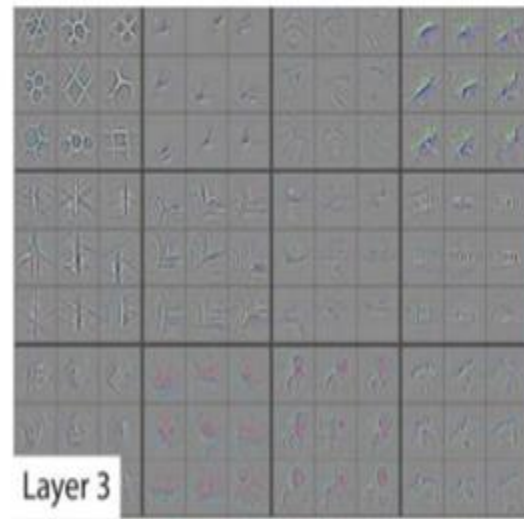


Regiões da imagem com respostas mais fortes

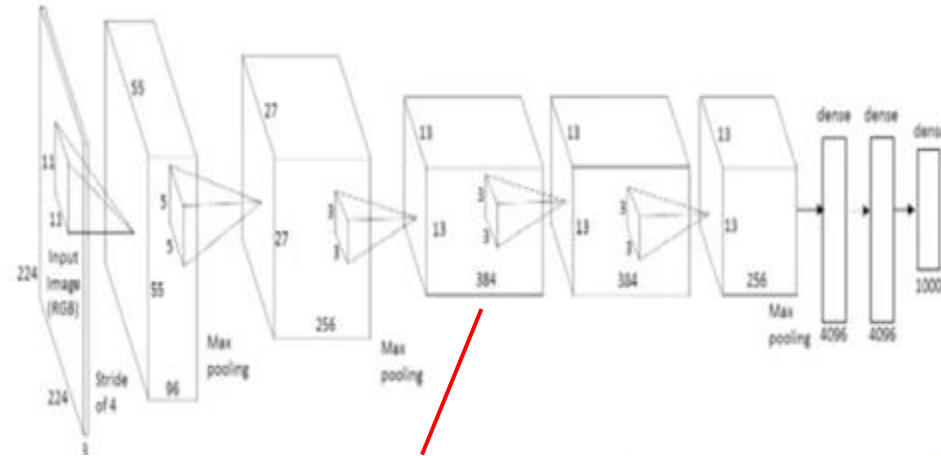
Redes Neurais Profundas



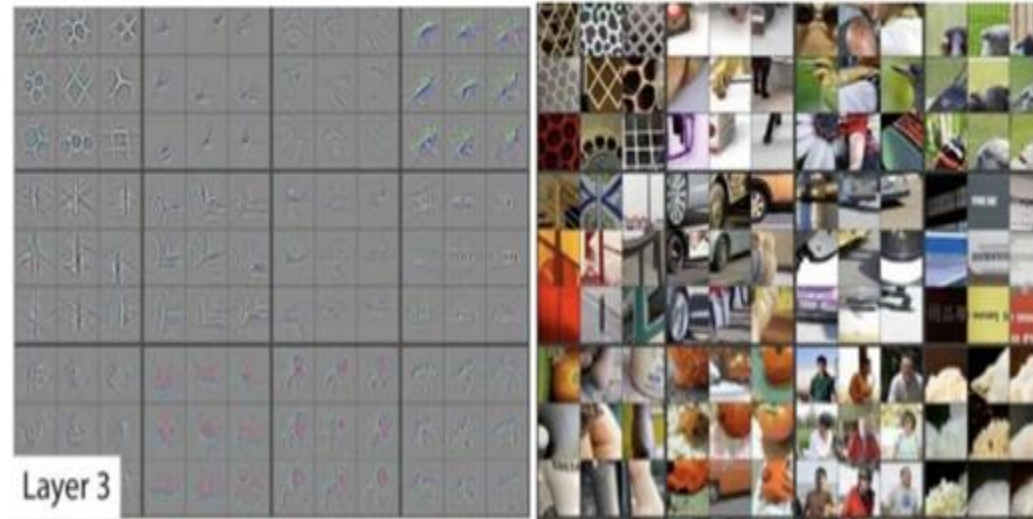
Resposta neural:



Redes Neurais Profundas

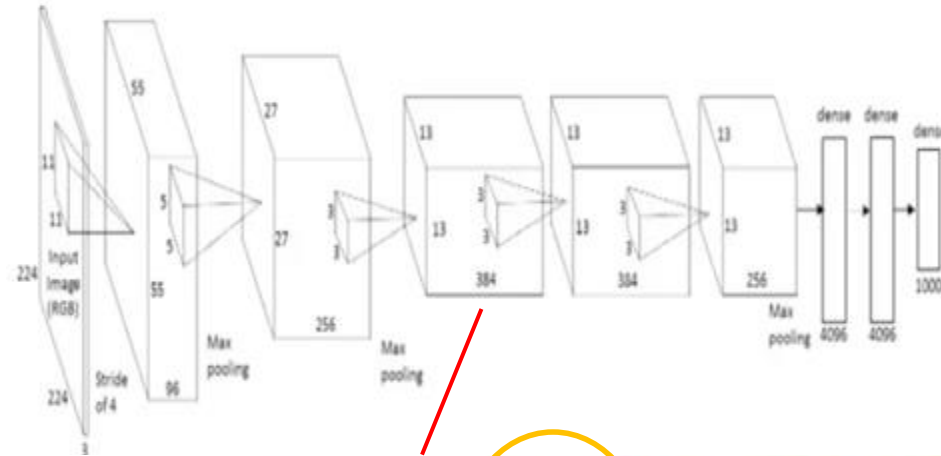


Resposta neural:

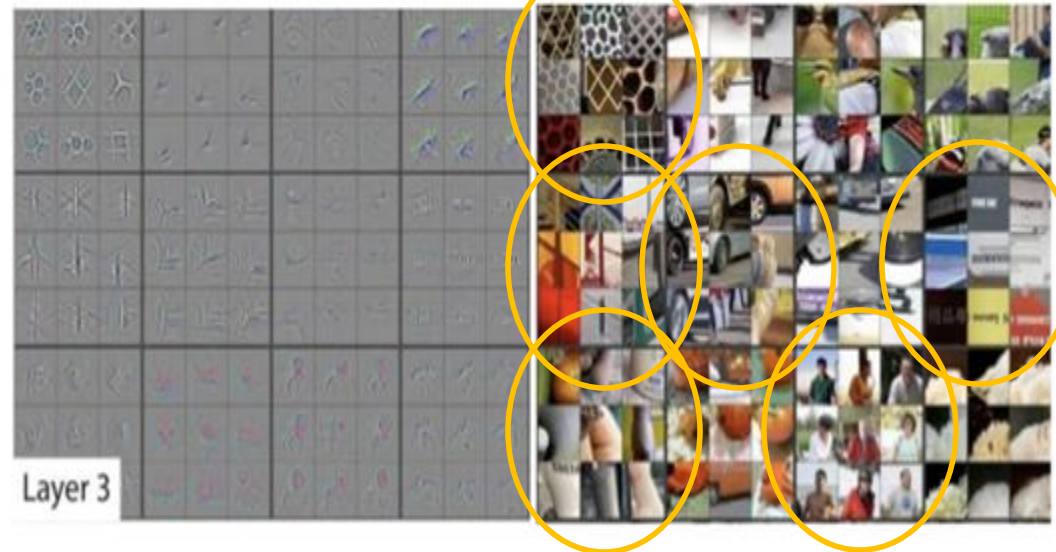


Regiões da imagem com respostas mais fortes

Redes Neurais Profundas

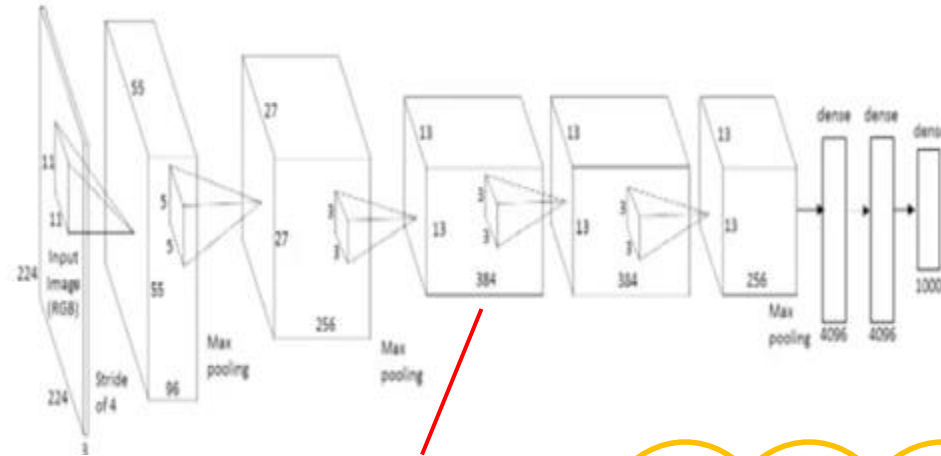


Resposta neural:



Regiões interpretáveis

Redes Neurais Profundas

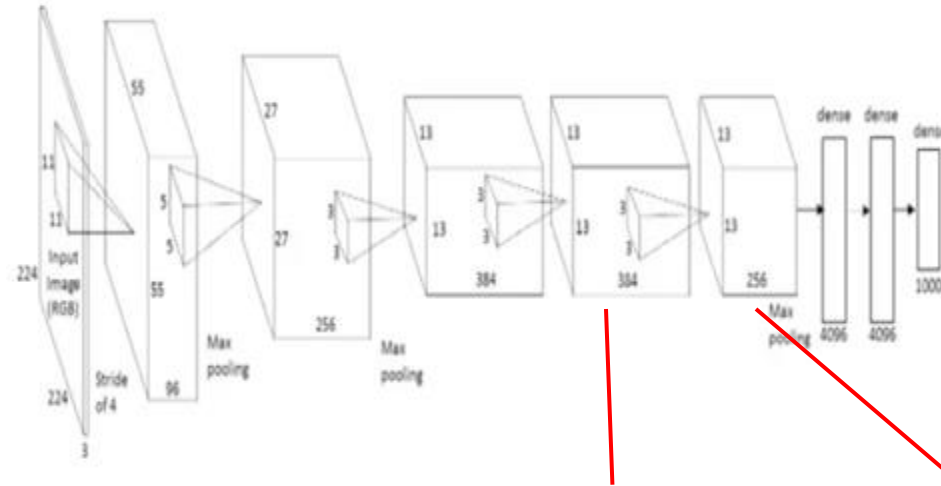


Resposta neural:

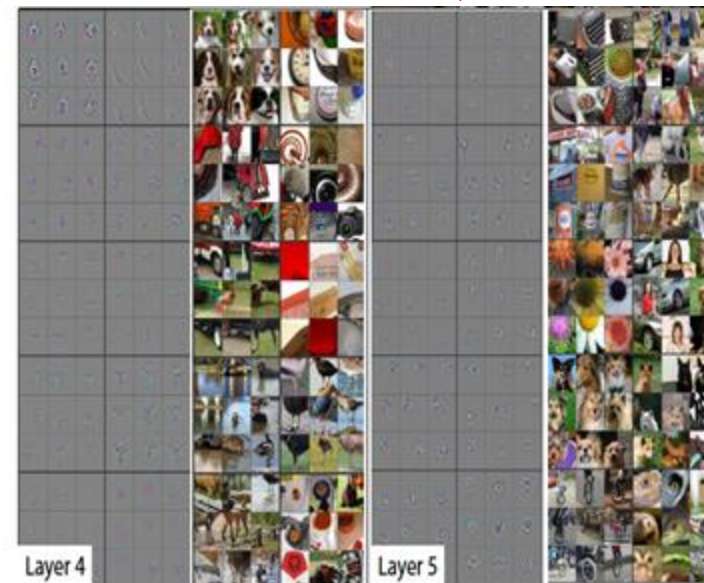


Porém nem todas ...

Redes Neurais Profundas



Resposta neural:

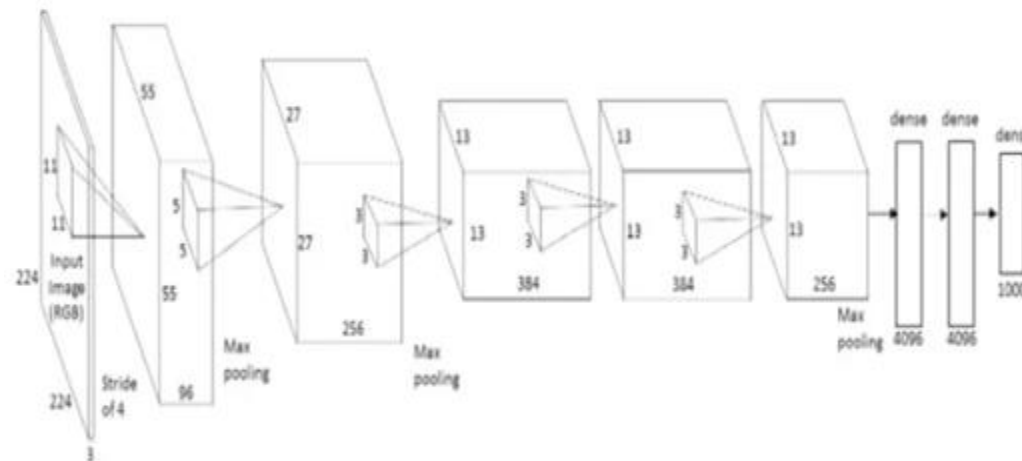


Redes Neurais Profundas

Redes profundas aprendem a resolver problemas gerais, a fim de resolver problemas específicos



1,4 M imagens



```
1: {0: 'tench, Tinca tinca',  
2: 1: 'goldfish, Carassius auratus',  
3: 2: 'great white shark, white shark, man-eater, man-eating  
4: 3: 'tiger shark, Galeocerdo cuvieri',  
5: 4: 'hammerhead, hammerhead shark',  
6: 5: 'electric ray, crampfish, numbfish, torpedo',  
7: 6: 'stingray',  
8: 7: 'cock',  
9: 8: 'hen',  
10: 9: 'ostrich, Struthio camelus',  
11: 10: 'brambling, Fringilla montifringilla',  
12: 11: 'goldfinch, Carduelis carduelis',  
13: 12: 'house finch, linnet, Cardopacus mexicanus',  
14: 13: 'junco, snowbird',  
15: 14: 'indigo bunting, indigo finch, indigo bird, Passerina',  
16: 15: 'robin, American robin, Turdus migratorius',  
17: 16: 'bulbul',  
18: 17: 'jay',  
19: 18: 'magpie',  
20: 19: 'chickadee',  
21: 20: 'water ouzel, dipper',  
22: 21: 'kite',  
23: 22: 'bald eagle, American eagle, Haliaeetus leucocephalus',  
24: 23: 'vulture',  
25: 24: 'great grey owl, great gray owl, Strix nebulosa',
```

⋮

1.000 classes

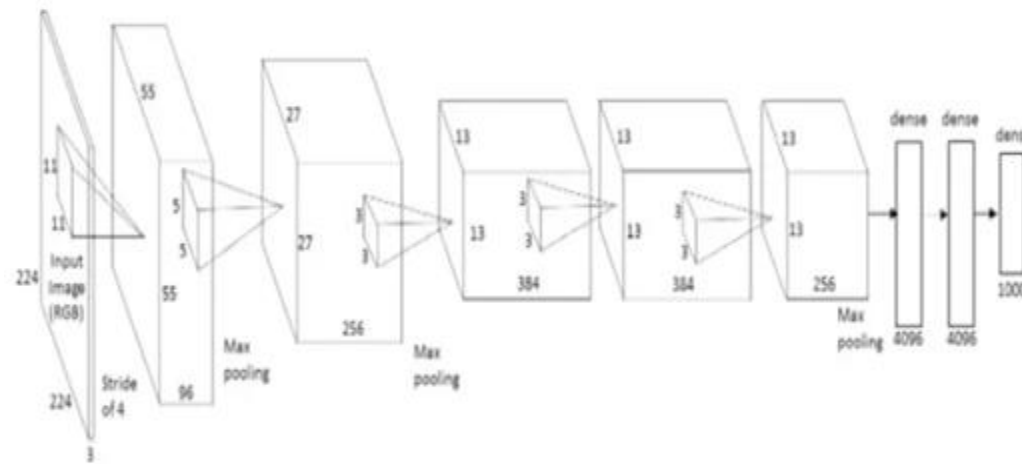
Redes Neurais Profundas

Redes profundas aprendem a resolver problemas gerais, a fim de resolver problemas específicos

Representações são aprendidas **apenas** a partir de imagens de entrada e rótulos de saída



1,4 M imagens



```
1 {0: 'tench, Tinca tinca',  
2 1: 'goldfish, Carassius auratus',  
3 2: 'great white shark, white shark, man-eater, man-eating',  
4 3: 'tiger shark, Galeocerdo cuvieri',  
5 4: 'hammerhead, hammerhead shark',  
6 5: 'electric ray, crampfish, numbfish, torpedo',  
7 6: 'stingray',  
8 7: 'cock',  
9 8: 'hen',  
10 9: 'ostrich, Struthio camelus',  
11 10: 'brambling, Fringilla montifringilla',  
12 11: 'goldfinch, Carduelis carduelis',  
13 12: 'house finch, linnet, Cardopacus mexicanus',  
14 13: 'junco, snowbird',  
15 14: 'indigo bunting, indigo finch, indigo bird, Passerina',  
16 15: 'robin, American robin, Turdus migratorius',  
17 16: 'bulbul',  
18 17: 'jay',  
19 18: 'magpie',  
20 19: 'chickadee',  
21 20: 'water ouzel, dipper',  
22 21: 'kite',  
23 22: 'bald eagle, American eagle, Haliaeetus leucocephalus',  
24 23: 'vulture',  
25 24: 'great grey owl, great gray owl, Strix nebulosa',
```

⋮

1.000 classes

Redes Neurais Profundas

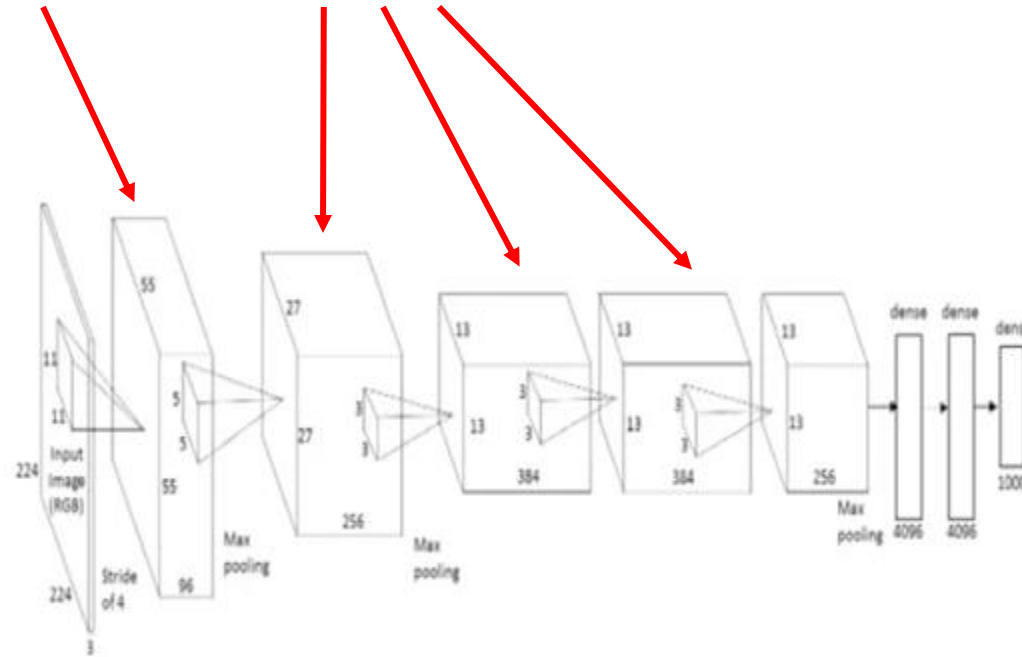
Redes profundas aprendem a resolver problemas gerais, a fim de resolver problemas específicos

Representações são aprendidas **apenas** a partir de imagens de entrada e rótulos de saída

Pesos **iniciais** e **intermediários** geralmente são independentes de tarefas



1,4 M imagens



```
1: {0: 'tench, Tinca tinca',  
2: 1: 'goldfish, Carassius auratus',  
3: 2: 'great white shark, white shark, man-eater, man-eating  
4: 3: 'tiger shark, Galeocerdo cuvieri',  
5: 4: 'hammerhead, hammerhead shark',  
6: 5: 'electric ray, crampfish, numbfish, torpedo',  
7: 6: 'stingray',  
8: 7: 'cock',  
9: 8: 'hen',  
10: 9: 'ostrich, Struthio camelus',  
11: 10: 'brambling, Fringilla montifringilla',  
12: 11: 'goldfinch, Carduelis carduelis',  
13: 12: 'house finch, linnet, Carpodacus mexicanus',  
14: 13: 'junco, snowbird',  
15: 14: 'indigo bunting, indigo finch, indigo bird, Passerina',  
16: 15: 'robin, American robin, Turdus migratorius',  
17: 16: 'bulbul',  
18: 17: 'jay',  
19: 18: 'magpie',  
20: 19: 'chickadee',  
21: 20: 'water ouzel, dipper',  
22: 21: 'kite',  
23: 22: 'bald eagle, American eagle, Haliaeetus leucocephalus',  
24: 23: 'vulture',  
25: 24: 'great grey owl, great gray owl, Strix nebulosa',
```

⋮

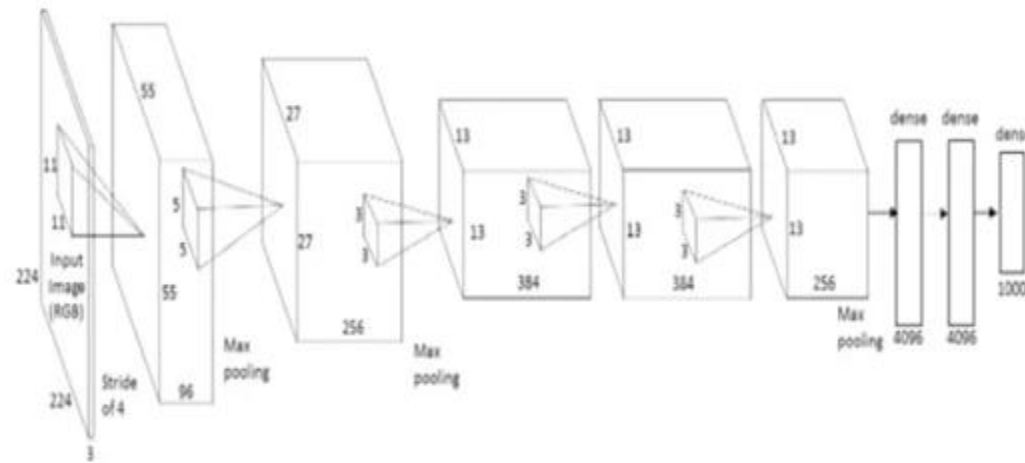
1.000 classes

Redes Neurais Profundas

CONSEQUÊNCIA: Pode-se **reutilizar** e/ou **ajustar** modelos treinados para tarefas diferentes com muito menos dados

Redes Neurais Profundas

CONSEQUÊNCIA: Pode-se **reutilizar** e/ou **ajustar** modelos treinados para tarefas diferentes com muito menos dados

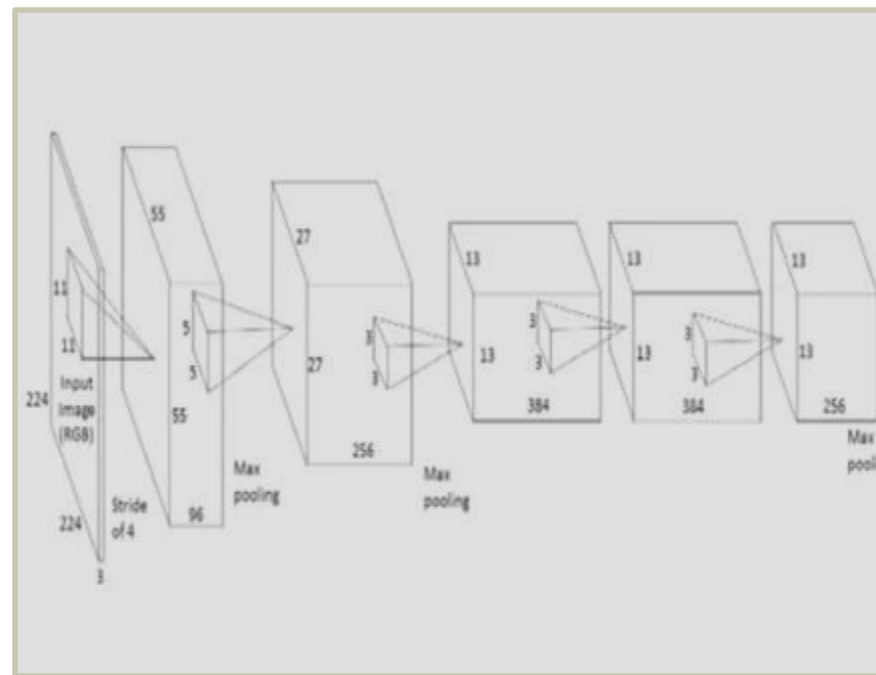


Classificação
de Imagens



Redes Neurais Profundas

CONSEQUÊNCIA: Pode-se **reutilizar** e/ou **ajustar** modelos treinados para tarefas diferentes com muito menos dados



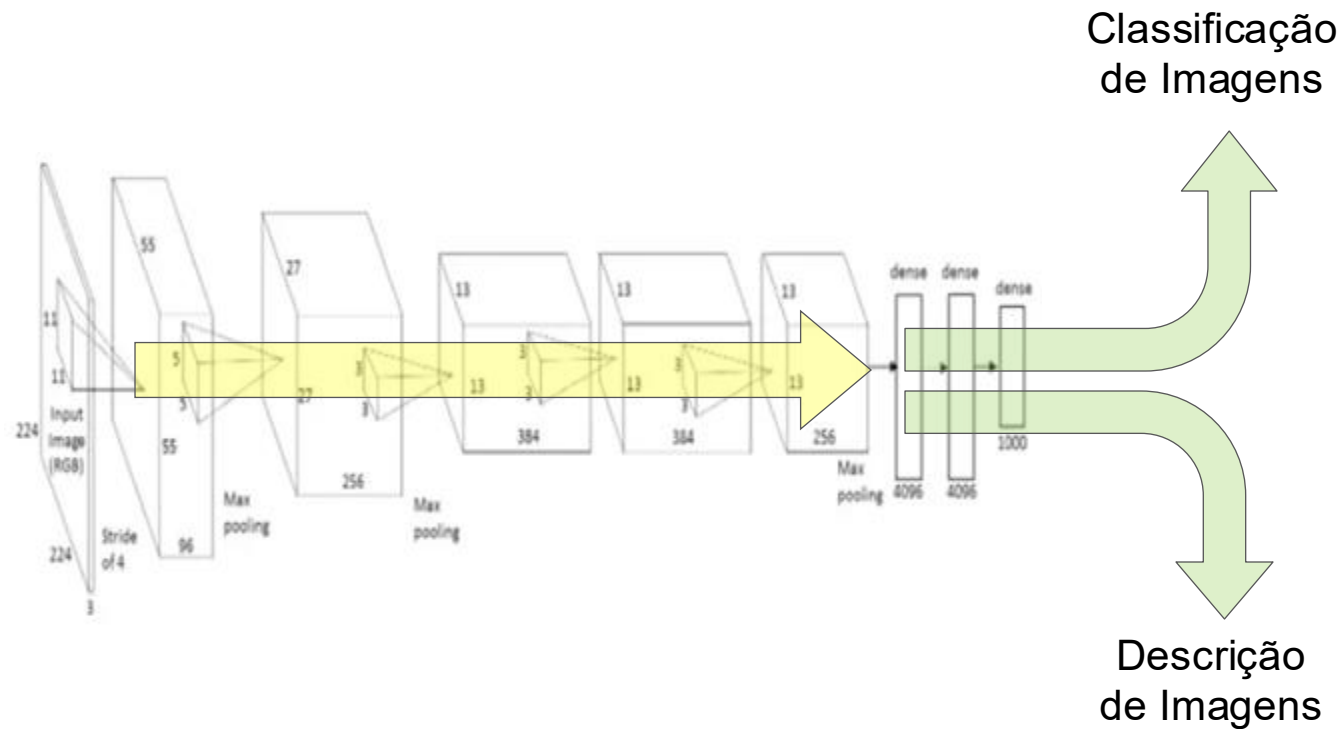
Congelar ou treinamento lento



Descrição
de Imagens

Redes Neurais Profundas

Aprendizagem multitarefa: Compartilhar os pesos entre tarefas geralmente melhora o desempenho de ambas



Redes Neurais Profundas

- As representações das camadas internas são aprendidas a partir do treinamento de tarefas feito apenas a partir das entradas e saídas ou ponta a ponta (“*end-to-end training*”)

Redes Neurais Profundas

- As representações das camadas internas são aprendidas a partir do treinamento de tarefas feito apenas a partir das entradas e saídas ou ponta a ponta (“*end-to-end training*”)
- A profundidade e a complexidade parecem ser limitadas apenas pela quantidade de dados necessários para se treinar sem “*overfitting*”. **Modelos mais complexos → melhor acurácia**

Redes Neurais Profundas

- As representações das camadas internas são aprendidas a partir do treinamento de tarefas feito apenas a partir das entradas e saídas ou ponta a ponta (“*end-to-end training*”)
- A profundidade e a complexidade parecem ser limitadas apenas pela quantidade de dados necessários para se treinar sem “*overfitting*”. **Modelos mais complexos → melhor acurácia**
- As representações de camadas internas iniciais são independentes de tarefas → facilidade em reutilizar modelos em diferentes aplicações sem a necessidade de grandes conjuntos de dados para treinamento

Redes Neurais Profundas

- As representações das camadas internas são aprendidas a partir do treinamento de tarefas feito apenas a partir das entradas e saídas ou ponta a ponta (“*end-to-end training*”)
- A profundidade e a complexidade parecem ser limitadas apenas pela quantidade de dados necessários para se treinar sem “*overfitting*”. **Modelos mais complexos → melhor acurácia**
- As representações de camadas internas iniciais são independentes de tarefas → facilidade em reutilizar modelos em diferentes aplicações sem a necessidade de grandes conjuntos de dados para treinamento
- Os blocos de construção (p.ex., tipos de camadas) pode ser padronizados entre diferentes ferramentas/bibliotecas e domínios de aplicação (imagem, fala, texto, robótica)... permitindo a transferência de abordagens, estratégias e “truques”

SGD – Treinamento de Rede Neural

SGD – *loop* principal:

```
while True:  
    data_batch = dataset.sample_data_batch()  
    loss = network.forward(data_batch)  
    dx = network.backward()  
    x += - learning_rate * dx
```

SGD – Treinamento de Rede Neural

SGD – *loop* principal:

```
while True:  
    data_batch = dataset.sample_data_batch()  
    loss = network.forward(data_batch)  
    dx = network.backward()  
    x += - learning_rate * dx
```



SGD – Treinamento de Rede Neural

SGD – *loop* principal:

```
while True:  
    data_batch = dataset.sample_data_batch()  
    loss = network.forward(data_batch)  
    dx = network.backward()  
    x += - learning_rate * dx
```



SGD – Treinamento de Rede Neural

SGD – *loop* principal:

```
while True:  
    data_batch = dataset.sample_data_batch()  
    loss = network.forward(data_batch)  
    dx = network.backward()  
    x += - learning_rate * dx
```



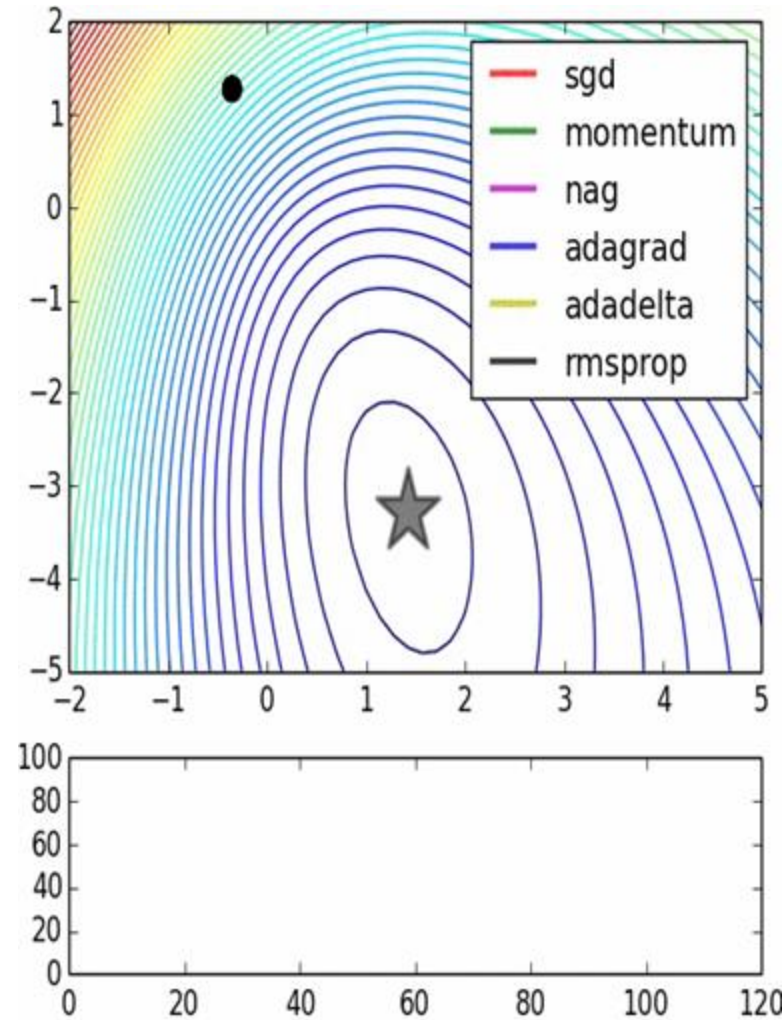
SGD – Treinamento de Rede Neural

SGD – *loop* principal:

```
while True:
    data_batch = dataset.sample_data_batch()
    loss = network.forward(data_batch)
    dx = network.backward()
    x += - learning_rate * dx
```

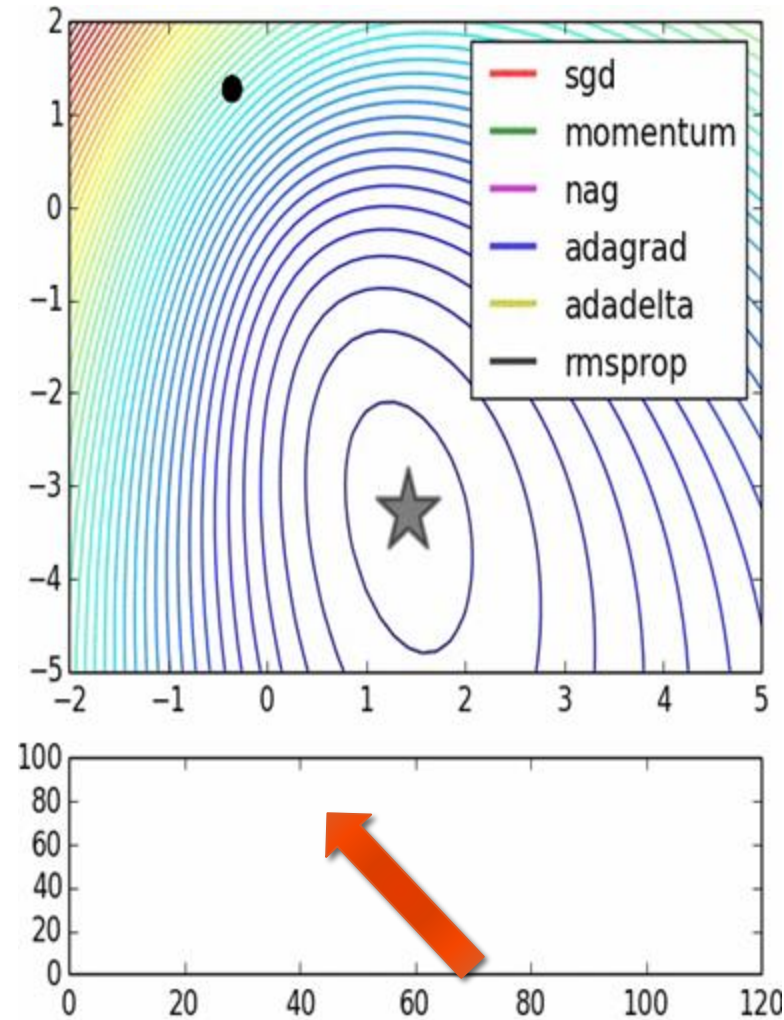
Atualização por meio de descida mais íngreme
(ou gradiente simples)

Vários Esquemas de Atualização de Pesos



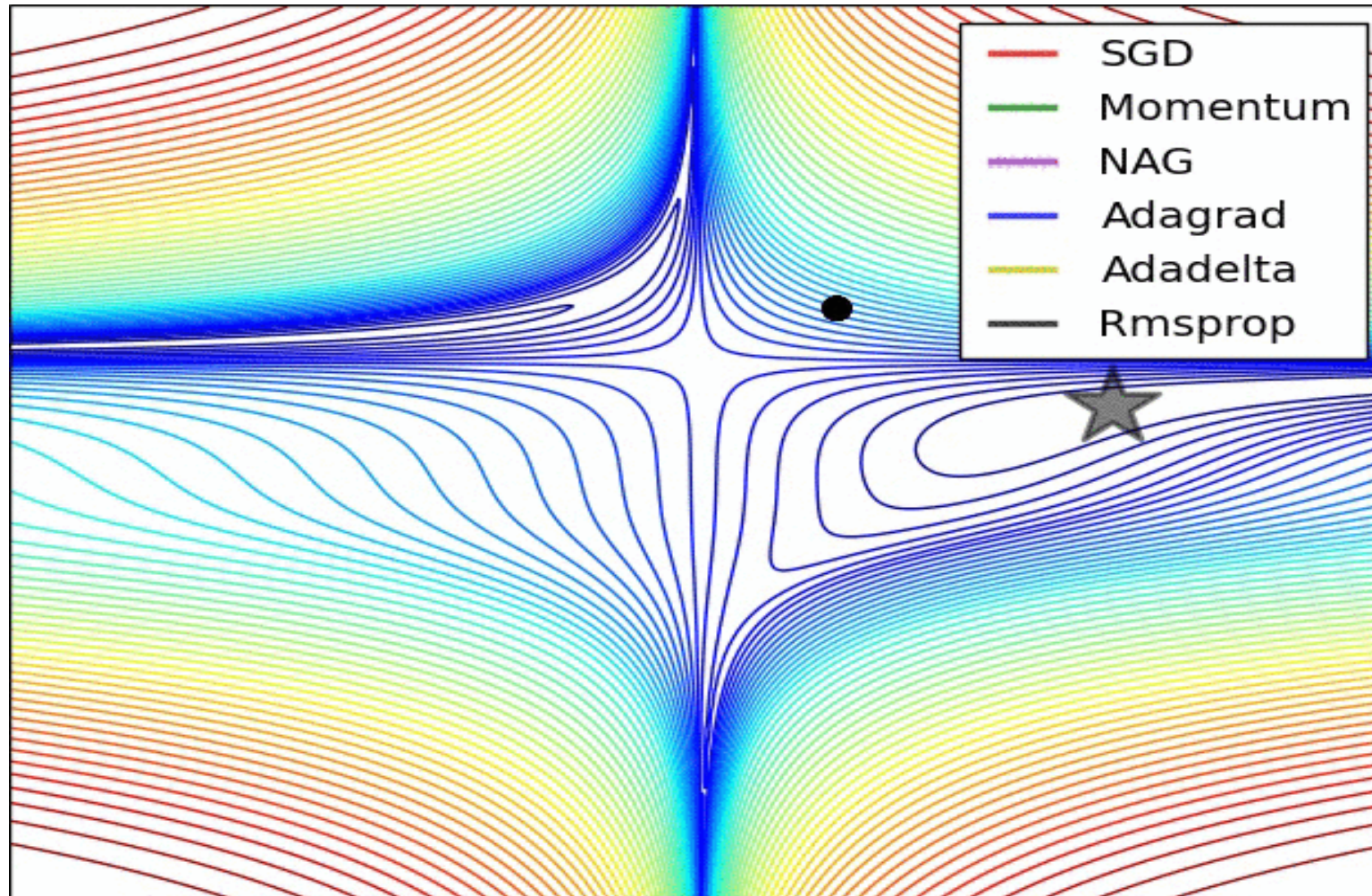
Crédito: Alec Radford, 2015

Vários Esquemas de Atualização de Pesos



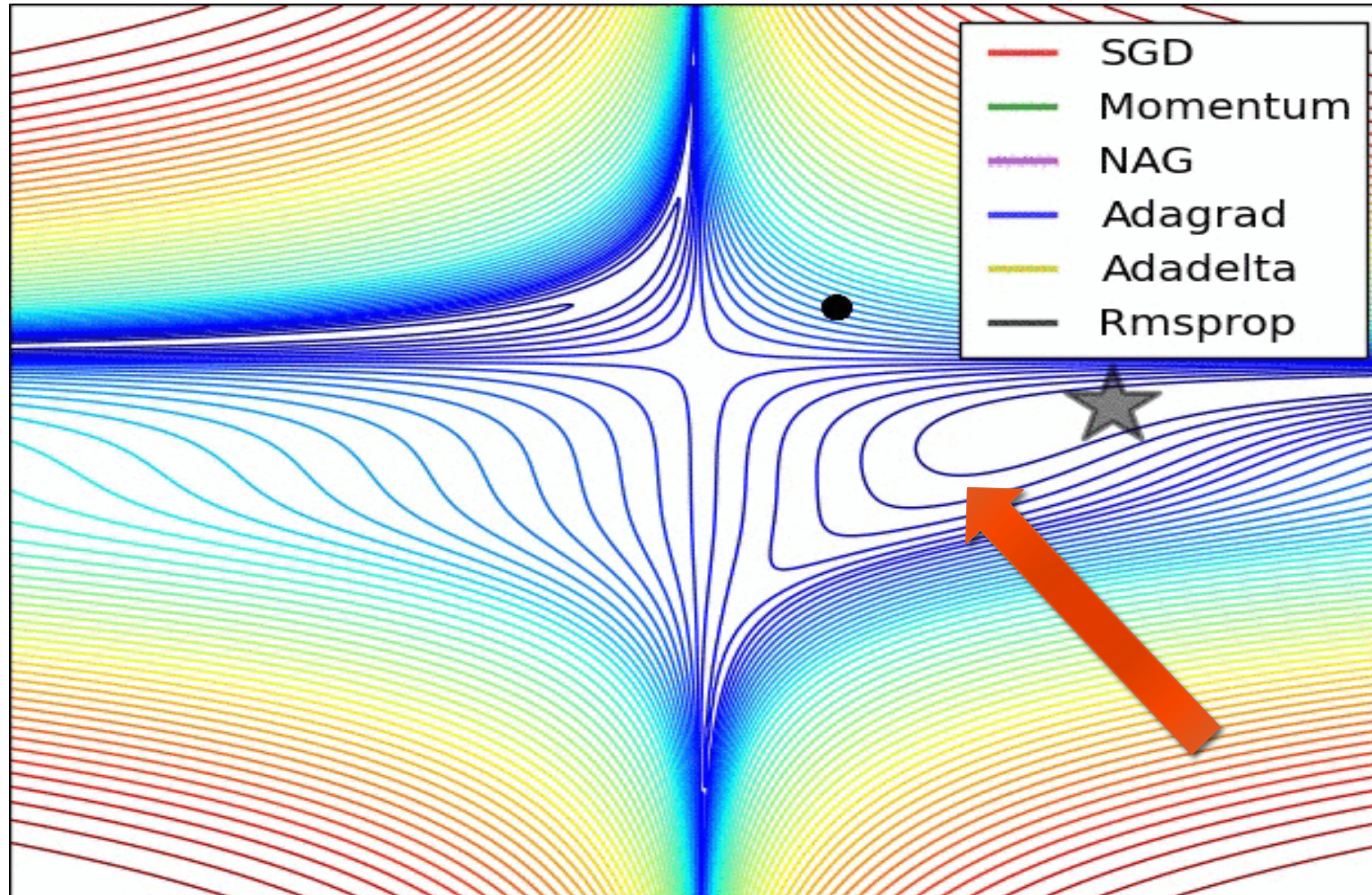
Crédito: Alec Radford, 2015

Vários Esquemas de Atualização de Pesos



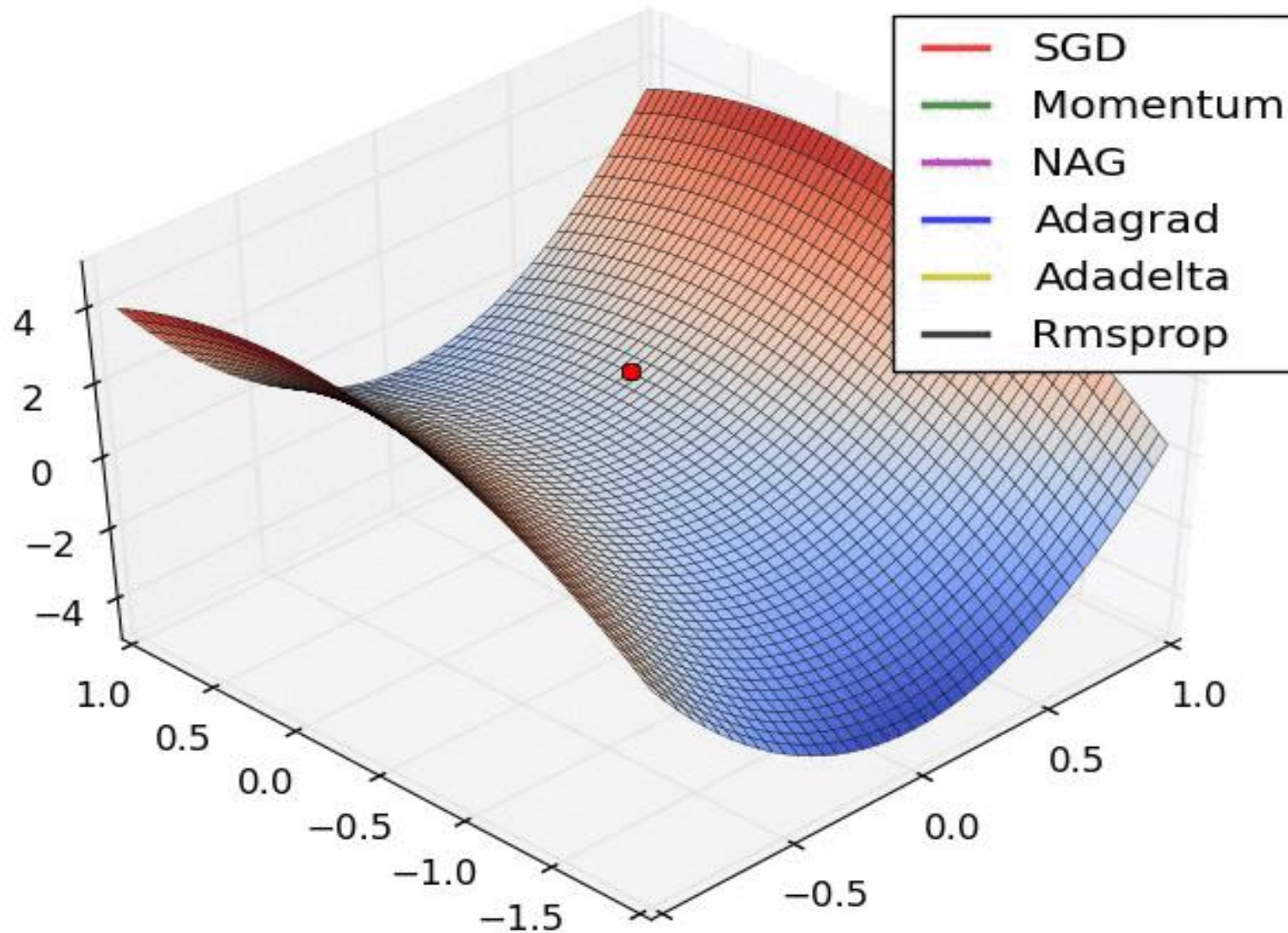
Crédito: Alec Radford, 2015

Vários Esquemas de Atualização de Pesos



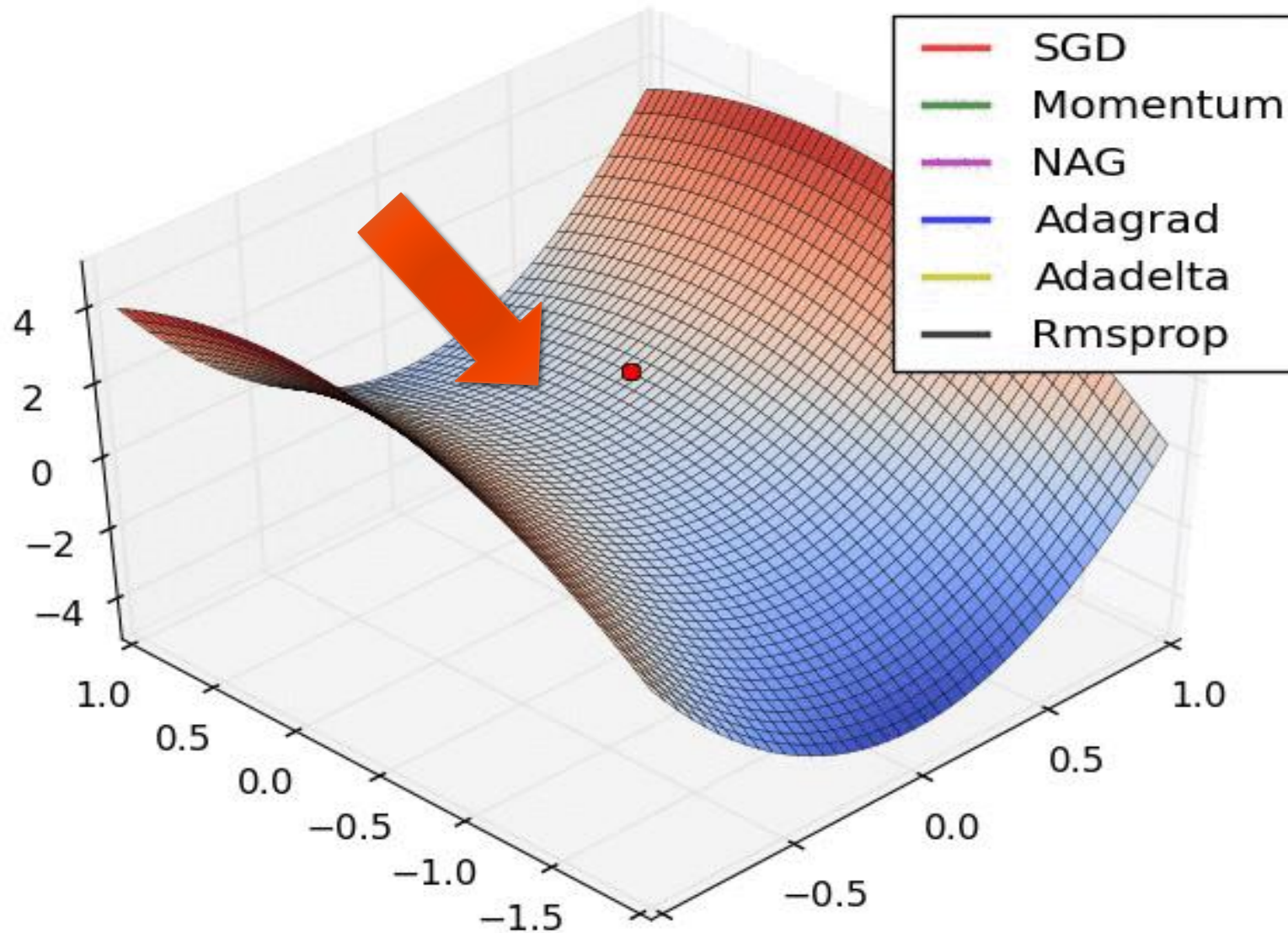
Crédito: Alec Radford, 2015

Vários Esquemas de Atualização de Pesos



Crédito: Alec Radford, 2015

Vários Esquemas de Atualização de Pesos



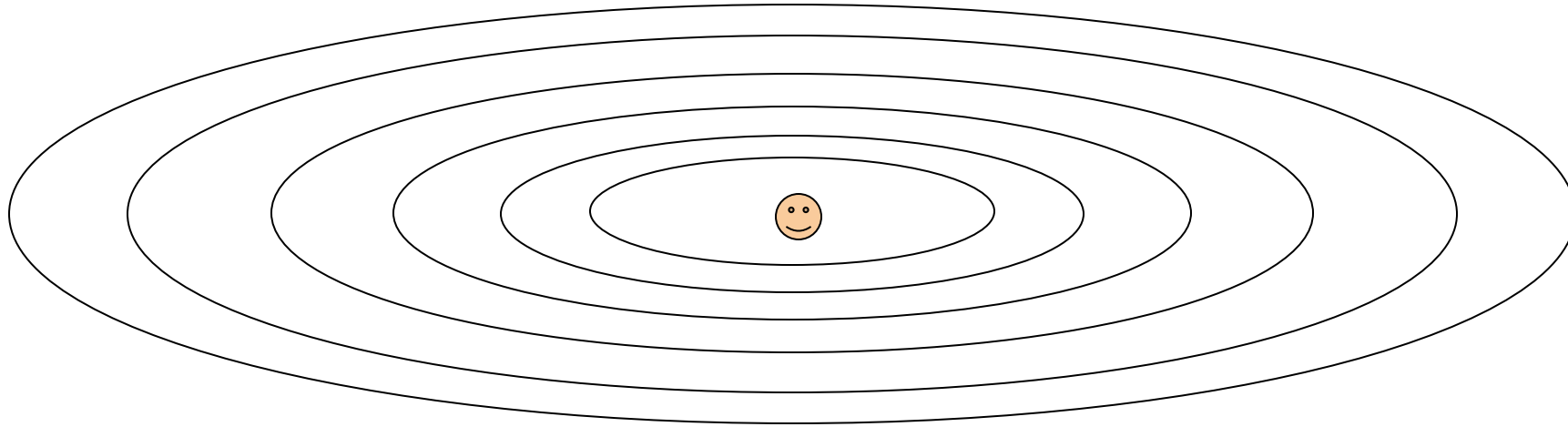
Crédito: Alec Radford, 2015

Problema com SGD

Suponha que a função de perda seja íngreme verticalmente, mas rasa horizontalmente

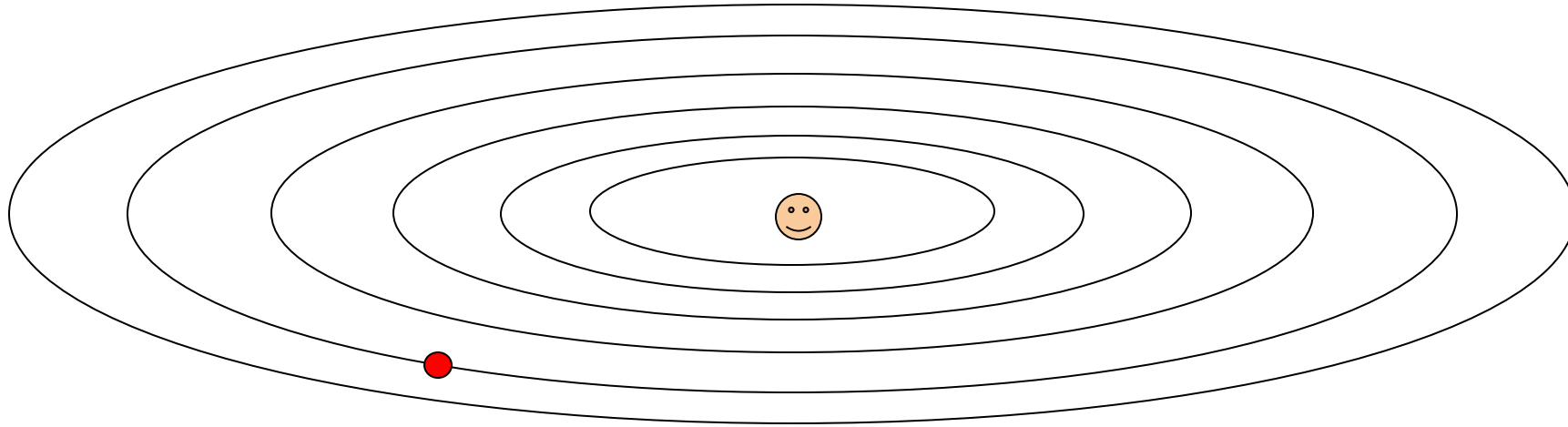
Problema com SGD

Suponha que a função de perda seja íngreme verticalmente, mas rasa horizontalmente



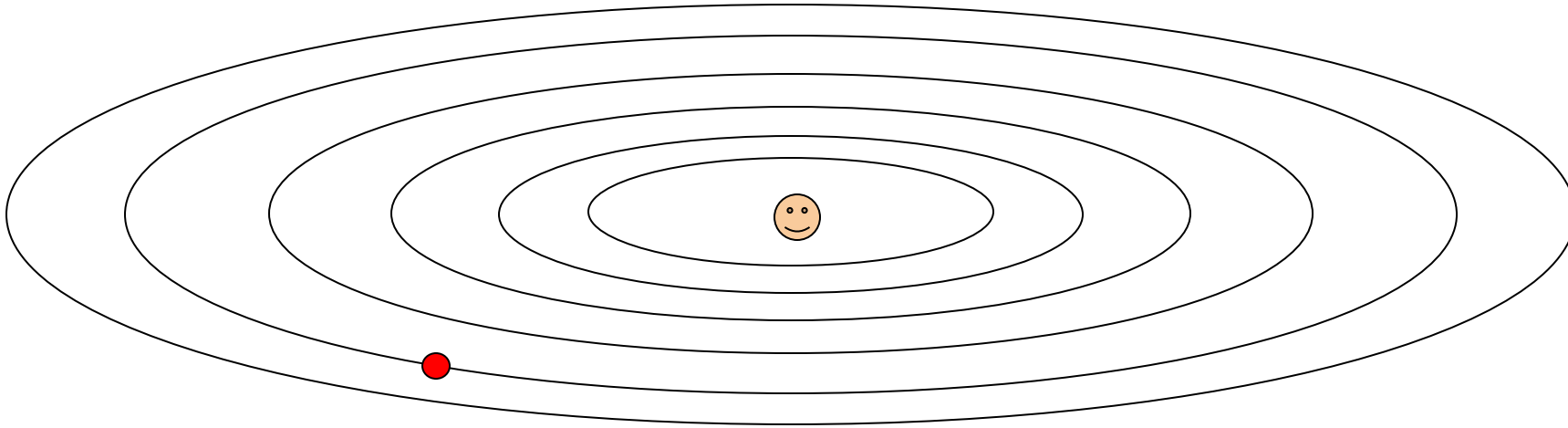
Problema com SGD

Suponha que a função de perda seja íngreme verticalmente, mas rasa horizontalmente



Problema com SGD

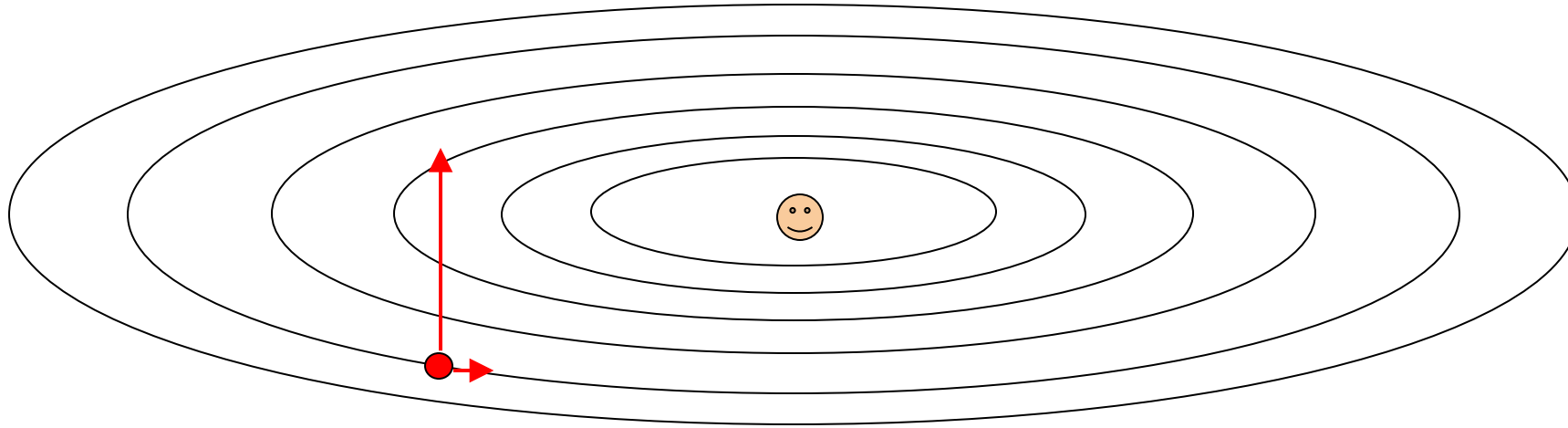
Suponha que a função de perda seja íngreme verticalmente, mas rasa horizontalmente



P: Qual é a trajetória que o SGD usa para alcançar o mínimo?

Problema com SGD

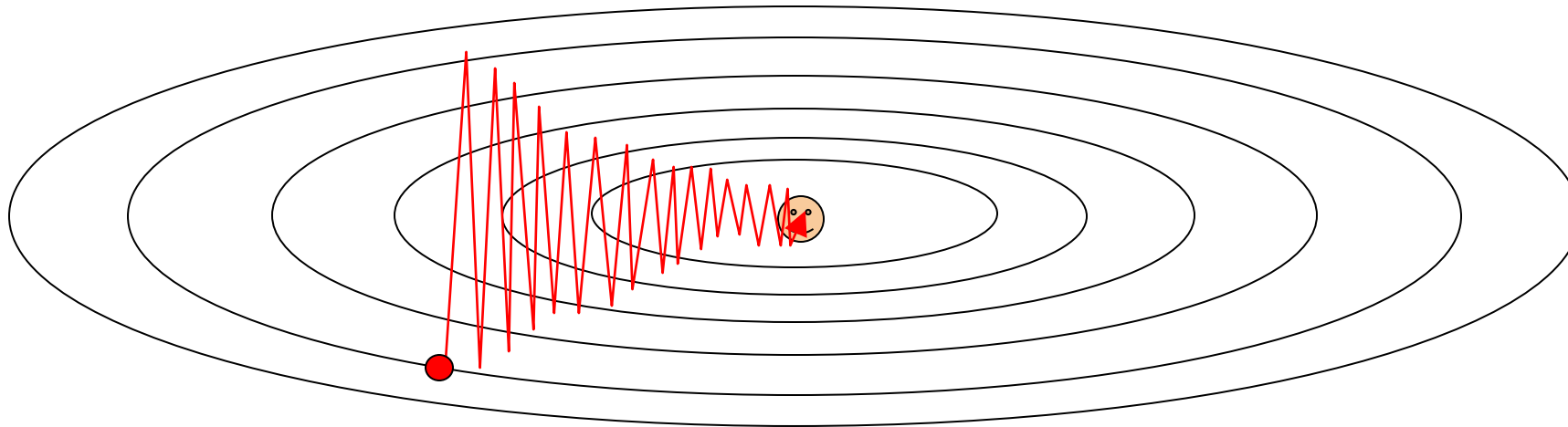
Suponha que a função de perda seja íngreme verticalmente, mas rasa horizontalmente



P: Qual é a trajetória que o SGD usa para alcançar o mínimo?

Problem with SGD

Suponha que a função de perda seja íngreme verticalmente, mas rasa horizontalmente



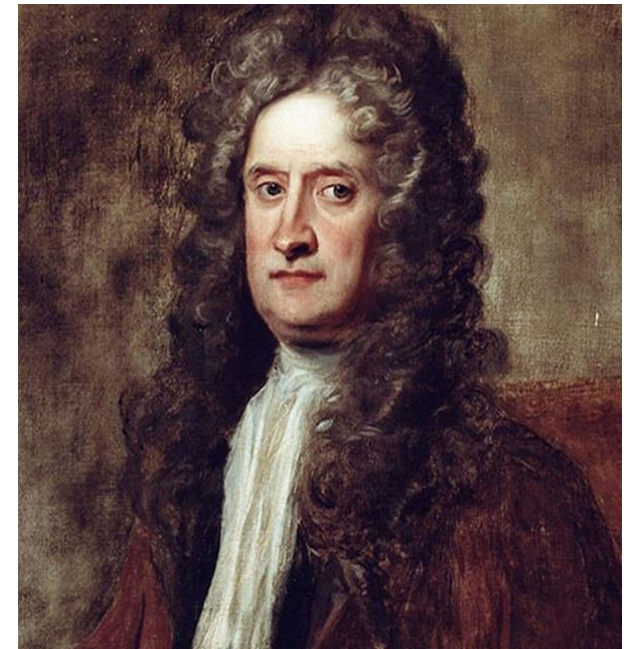
P: Qual é a trajetória que o SGD usa para alcançar o mínimo?

Progresso muito lento na horizontal, “zig-zag” na vertical

SGD com *Momentum*

“Todo corpo persiste em seu estado de repouso ou de se mover uniformemente para a frente, exceto na medida em que é obrigado a mudar de estado por uma força impressa.”

– Isaac Newton

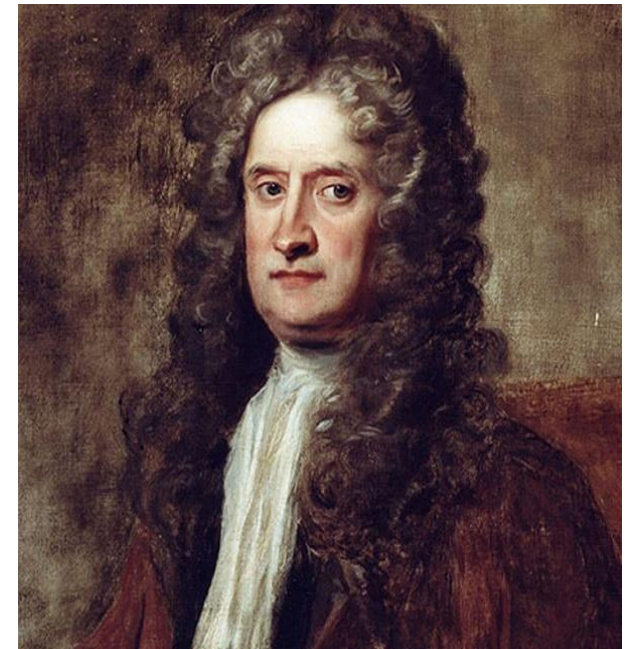


SGD com *Momentum*

“Todo corpo persiste em seu estado de repouso ou de se mover uniformemente para a frente, exceto na medida em que é obrigado a mudar de estado por uma força impressa.”

– Isaac Newton

A “memória” do objeto de seu estado de movimento é ***momentum***



SGD com *Momentum*

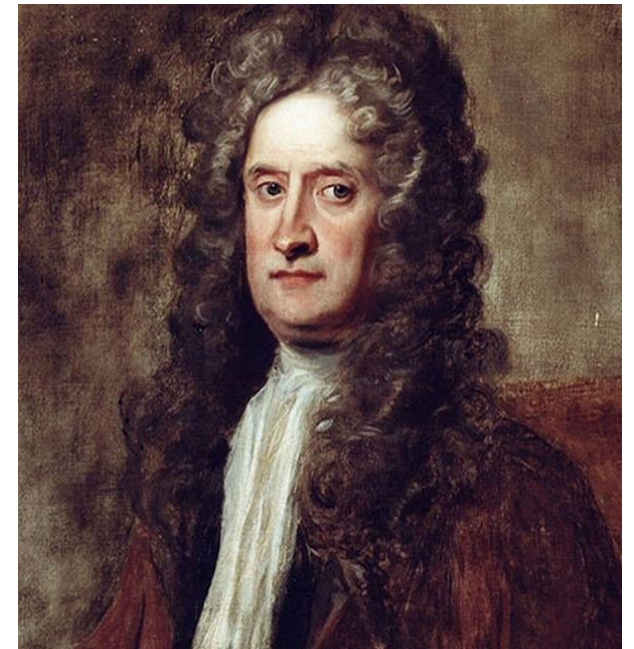
“Todo corpo persiste em seu estado de repouso ou de se mover uniformemente para a frente, exceto na medida em que é obrigado a mudar de estado por uma força impressa.”

– Isaac Newton

A “memória” do objeto de seu estado de movimento é ***momentum***

Como é comumente usado em aprendizagem profunda, o parâmetro “*momentum*” é na verdade a “taxa de decaimento do *momentum* por minibatch”

(E seu análogo físico seria a viscosidade)



SGD com *Momentum*

Regra de atualização do “*momentum*” para o passo t :

$$p^{(t+1)} = \mu p^{(t)} - \alpha g^{(t)}$$

em que

- $p^{(t)}$ representa o “*momentum*”,

SGD com *Momentum*

Regra de atualização do “*momentum*” para o passo t :

$$p^{(t+1)} = \mu p^{(t)} - \alpha g^{(t)}$$

em que

- $p^{(t)}$ representa o “*momentum*”,
- $\mu \in [0,1]$ é a constante de “*momentum*”,

SGD com *Momentum*

Regra de atualização do “*momentum*” para o passo t :

$$p^{(t+1)} = \mu p^{(t)} - \alpha g^{(t)}$$

em que

- $p^{(t)}$ representa o “*momentum*”,
- $\mu \in [0,1]$ é a constante de “*momentum*”,
- $g^{(t)}$ é o gradiente do *minibatch*, isto é,

$$g^{(t)} = \nabla_W L(W^{(t)}),$$

SGD com *Momentum*

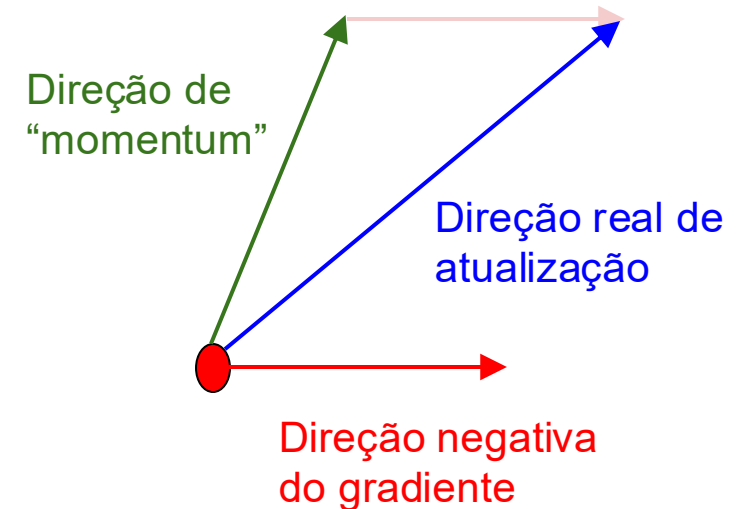
Regra de atualização do “*momentum*” para o passo t :

$$p^{(t+1)} = \mu p^{(t)} - \alpha g^{(t)}$$

em que

- $p^{(t)}$ representa o “*momentum*”,
- $\mu \in [0,1]$ é a constante de “*momentum*”,
- $g^{(t)}$ é o gradiente do *minibatch*, isto é,
$$g^{(t)} = \nabla_W L(W^{(t)}),$$
 e
- α é a taxa de aprendizado

Atualização pelo “*Momentum*”



SGD com *Momentum*

Regra de atualização do “*momentum*” para o passo t :

$$p^{(t+1)} = \mu p^{(t)} - \alpha g^{(t)}$$

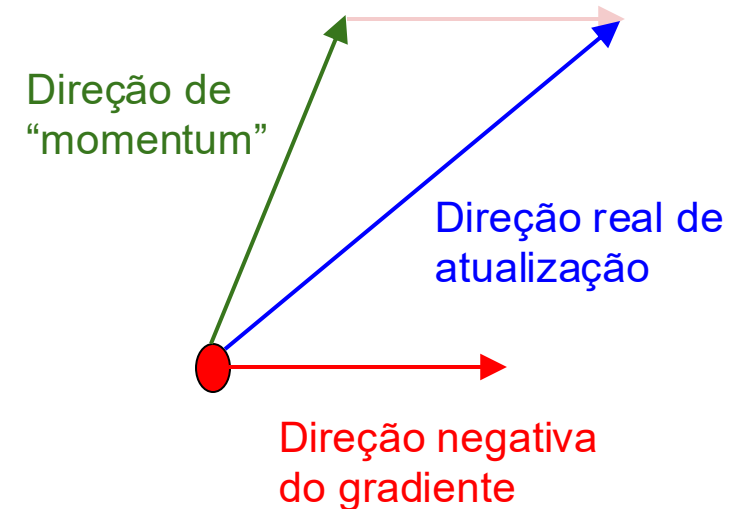
em que

- $p^{(t)}$ representa o “*momentum*”,
- $\mu \in [0,1]$ é a constante de “*momentum*”,
- $g^{(t)}$ é o gradiente do *minibatch*, isto é,
$$g^{(t)} = \nabla_W L(W^{(t)}),$$
 e
- α é a taxa de aprendizado

A atualização de pesos é dados por:

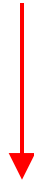
$$W^{(t+1)} = W^{(t)} + p^{(t+1)}$$

Atualização pelo “*Momentum*”



Atualização pelo “*Momentum*”

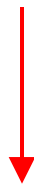
```
# Gradient descent update  
x += - learning_rate * dx
```



```
# Momentum update  
v = mu * v - learning_rate * dx # integrate velocity  
x += v # integrate position
```

Atualização pelo “*Momentum*”

```
# Gradient descent update  
x += - learning_rate * dx
```

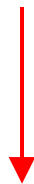


```
# Momentum update  
v = mu * v - learning_rate * dx # integrate velocity  
x += v # integrate position
```

- Interpretação física como uma bola descendo pela função de perda + atrito (coeficiente μ)
- μ = geralmente $\sim 0,5$; $0,9$ ou $0,99$ (às vezes ajustado ao longo do tempo, p.ex. de $0,5 \rightarrow 0,99$)

Atualização pelo “*Momentum*”

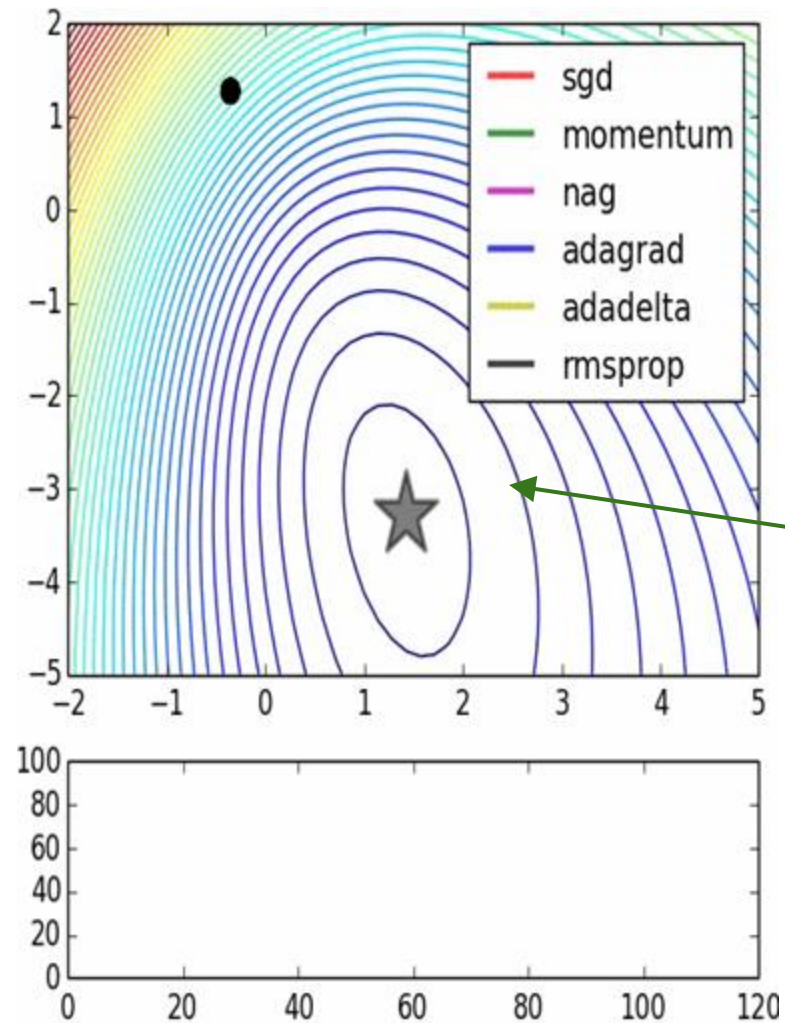
```
# Gradient descent update  
x += - learning_rate * dx
```



```
# Momentum update  
v = mu * v - learning_rate * dx # integrate velocity  
x += v # integrate position
```

- Interpretação física como uma bola descendo pela função de perda + atrito (coeficiente μ)
- μ = geralmente $\sim 0,5$; $0,9$ ou $0,99$ (às vezes ajustado ao longo do tempo, p.ex. de $0,5 \rightarrow 0,99$)
- Permite que a velocidade “se acumule” nas direções rasas
- Reduz a velocidade nas direções íngremes devido à mudança rápida de sinal

SGD × SGD+*Momentum*



Observe que o “momentum” ultrapassar a meta, mas, em geral, chega ao mínimo muito mais rápido

RMSProp

[\[Hinton et al., 2012\]](#)

Os gradientes podem variar muito, embora os parâmetros geralmente tenham a mesma escala

RMSProp

[Hinton et al., 2012]

Os gradientes podem variar muito, embora os parâmetros geralmente tenham a mesma escala

RMSProp (*Root Mean Squared Propagation*) ajusta os valores de gradientes pelo inverso de uma média móvel

RMSProp

[Hinton et al., 2012]

Os gradientes podem variar muito, embora os parâmetros geralmente tenham a mesma escala

RMSProp (*Root Mean Squared Propagation*) ajusta os valores de gradientes pelo inverso de uma média móvel

Define-se

$$s^{(t+1)} = \beta s^{(t)} + (1 - \beta)(g^{(t)})^2$$

em que:

- $s^{(t)}$ é o gradiente quadrático médio (média móvel) no passo t ,

RMSProp

[Hinton et al., 2012]

Os gradientes podem variar muito, embora os parâmetros geralmente tenham a mesma escala

RMSProp (*Root Mean Squared Propagation*) ajusta os valores de gradientes pelo inverso de uma média móvel

Define-se

$$s^{(t+1)} = \beta s^{(t)} + (1 - \beta)(g^{(t)})^2$$

em que:

- $s^{(t)}$ é o gradiente quadrático médio (média móvel) no passo t ,
- $\beta \in [0,1]$ é o fator de decaimento da média móvel,

RMSProp

[Hinton et al., 2012]

Os gradientes podem variar muito, embora os parâmetros geralmente tenham a mesma escala

RMSProp (*Root Mean Squared Propagation*) ajusta os valores de gradientes pelo inverso de uma média móvel

Define-se

$$s^{(t+1)} = \beta s^{(t)} + (1 - \beta)(g^{(t)})^2$$

quadrado elemento a elemento de $g^{(t)}$,
assim $s^{(t)}$ possui as mesmas dimensões que $g^{(t)}$

em que:

- $s^{(t)}$ é o gradiente quadrático médio (média móvel) no passo t ,
- $\beta \in [0,1]$ é o fator de decaimento da média móvel,
- $g^{(t)}$ é o gradiente do *minibatch* no passo t

RMSProp

[Hinton et al., 2012]

Os gradientes podem variar muito, embora os parâmetros geralmente tenham a mesma escala

RMSProp (*Root Mean Squared Propagation*) ajusta os valores de gradientes pelo inverso de uma média móvel

Define-se

$$s^{(t+1)} = \beta s^{(t)} + (1 - \beta)(g^{(t)})^2$$

quadrado elemento a elemento de $g^{(t)}$,
assim $s^{(t)}$ possui as mesmas dimensões que $g^{(t)}$

em que:

- $s^{(t)}$ é o gradiente quadrático médio (média móvel) no passo t ,
- $\beta \in [0,1]$ é o fator de decaimento da média móvel,
- $g^{(t)}$ é o gradiente do *minibatch* no passo t

A regra de atualização do **RMSProp** é dada por:

$$W^{(t+1)} = W^{(t)} - \alpha \frac{g^{(t)}}{\sqrt{s^{(t)}}} \rightarrow \text{divisão elemento a elemento!}$$

ADAGRAD

[Duchi et al., 2011]

ADAGRAD (*AD*Aptive *GR*ADient) é semelhante ao RMSProp, porém utiliza a **soma acumulada** dos quadrados dos gradientes

ADAGRAD

[Duchi et al., 2011]

ADAGRAD (*AD*Aptive *GR*ADient) é semelhante ao RMSProp, porém utiliza a **soma acumulada** dos quadrados dos gradientes

Define-se

$$c^{(t)} = \sum_{j=1}^t (g^{(j)})^2$$

→ quadrado elemento a elemento de $g^{(t)}$,
assim $c^{(t)}$ possui as mesmas dimensões que $g^{(t)}$

ADAGRAD

[Duchi et al., 2011]

ADAGRAD (*AD*Aptive *GR*ADient) é semelhante ao RMSProp, porém utiliza a **soma acumulada** dos quadrados dos gradientes

Define-se

$$c^{(t)} = \sum_{j=1}^t (g^{(j)})^2$$

quadrado elemento a elemento de $g^{(t)}$,
assim $c^{(t)}$ possui as mesmas dimensões que $g^{(t)}$

Assim, $c^{(t)}$ representa o gradiente quadrático **acumulado** no passo t

ADAGRAD

[Duchi et al., 2011]

ADAGRAD (*AD*Aptive *GR*ADient) é semelhante ao RMSProp, porém utiliza a **soma acumulada** dos quadrados dos gradientes

Define-se

$$c^{(t)} = \sum_{j=1}^t (g^{(j)})^2$$

→ quadrado elemento a elemento de $g^{(t)}$,
assim $c^{(t)}$ possui as mesmas dimensões que $g^{(t)}$

Assim, $c^{(t)}$ representa o gradiente quadrático **acumulado** no passo t

A regra de atualização do **ADAGRAD** é dada por:

$$W^{(t+1)} = W^{(t)} - \alpha \frac{g^{(t)}}{\sqrt{c^{(t)}}}$$

→ divisão elemento a elemento!

OBS: $c^{(t)}$ tende a crescer linearmente em função de t , assim o ADAGRAD reduz sua taxa de aprendizado efetiva ao longo do tempo em $1/\sqrt{t}$

RMSprop × ADAGRAD

```
# RMSProp
cache = decay_rate * cache + (1 - decay_rate) * dx**2
x += - learning_rate * dx / (np.sqrt(cache) + 1e-7)
```

```
# Adagrad update
cache += dx**2
x += - learning_rate * dx / (np.sqrt(cache) + 1e-7)
```

RMSprop × ADAGRAD

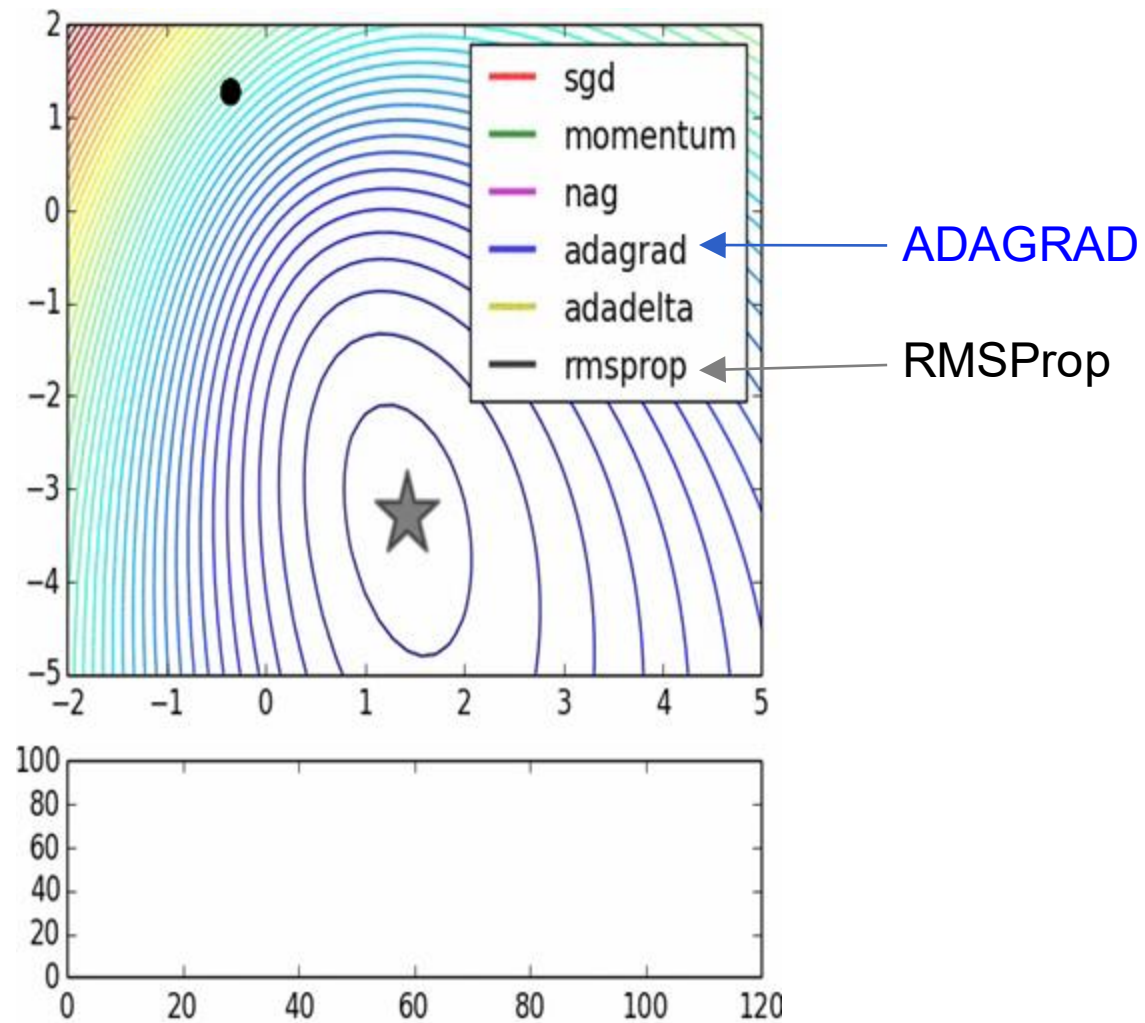
```
# RMSProp
cache = decay_rate * cache + (1 - decay_rate) * dx**2
x += - learning_rate * dx / (np.sqrt(cache) + 1e-7)
```

```
# Adagrad update
cache += dx**2
x += - learning_rate * dx / (np.sqrt(cache) + 1e-7)
```

Diferença



RMSProp × ADAGRAD



Crédito: Alec Radford, 2015

RMSProp × ADAGRAD

- Como ambos normalizam magnitudes de gradiente, o RMSProp e o ADAGRAD funcionam muito bem em conjuntos de dados com grande variação na magnitude de gradientes

RMSProp × ADAGRAD

- Como ambos normalizam magnitudes de gradiente, o RMSProp e o ADAGRAD funcionam muito bem em conjuntos de dados com grande variação na magnitude de gradientes
- O exemplo mais comum de aplicação são dados textuais, nos quais os gradientes variam de 4 a 5 ordens de magnitude

RMSProp × ADAGRAD

- Como ambos normalizam magnitudes de gradiente, o RMSProp e o ADAGRAD funcionam muito bem em conjuntos de dados com grande variação na magnitude de gradientes
- O exemplo mais comum de aplicação são dados textuais, nos quais os gradientes variam de 4 a 5 ordens de magnitude
- O uso do RMSProp / ADAGRAD pode acelerar o aprendizado de modelos de texto em 2-3 ordens de magnitude

RMSProp × ADAGRAD

- Como ambos normalizam magnitudes de gradiente, o RMSProp e o ADAGRAD funcionam muito bem em conjuntos de dados com grande variação na magnitude de gradientes
- O exemplo mais comum de aplicação são dados textuais, nos quais os gradientes variam de 4 a 5 ordens de magnitude
- O uso do RMSProp / ADAGRAD pode acelerar o aprendizado de modelos de texto em 2-3 ordens de magnitude
- Porém, menos eficaz quando existem fortes dependências entre as características

RMSProp × ADAGRAD

- O RMSProp é uma abordagem heurística, enquanto o ADAGRAD possui limites formais na sua taxa de convergência, embora apenas para problemas convexos

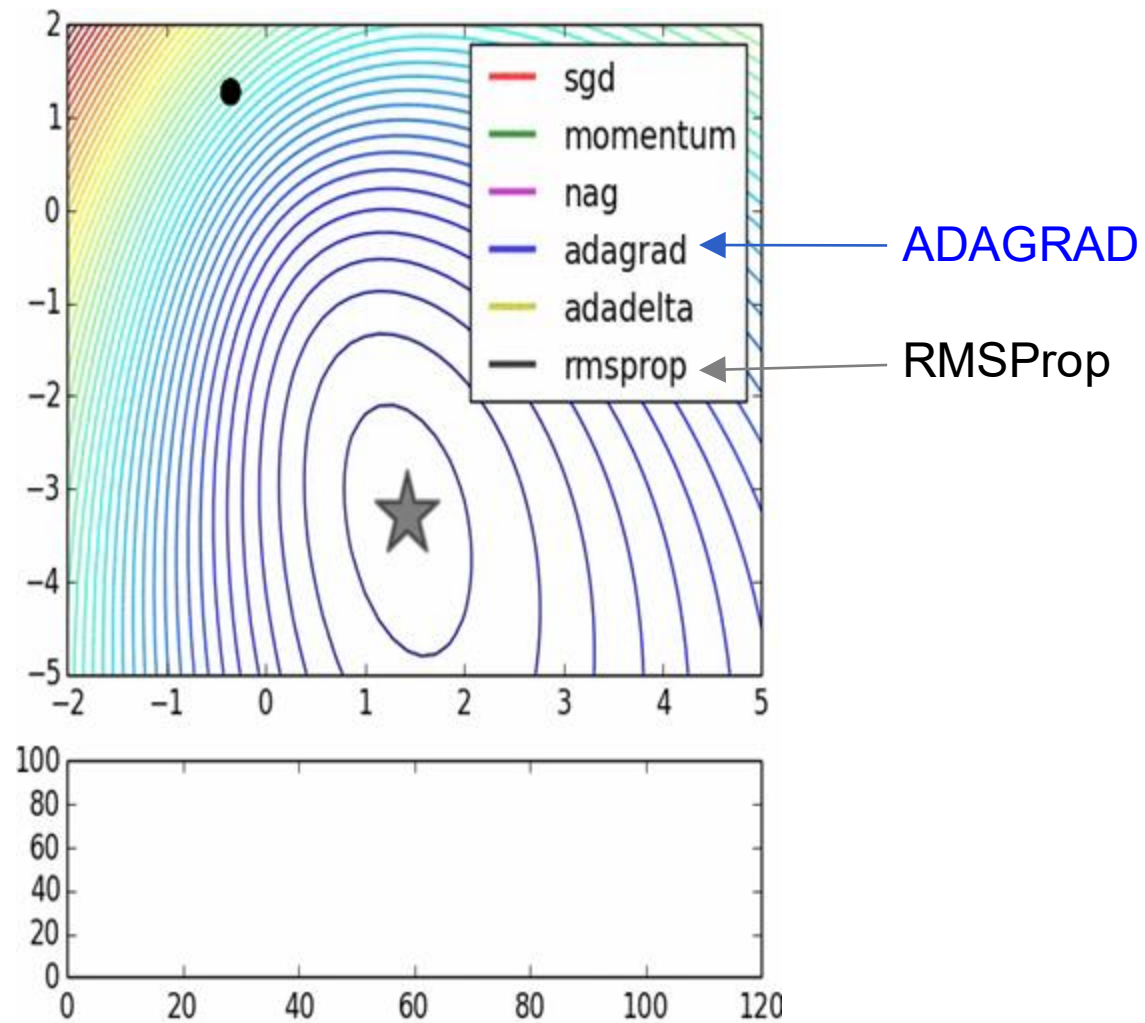
RMSProp × ADAGRAD

- O RMSProp é uma abordagem heurística, enquanto o ADAGRAD possui limites formais na sua taxa de convergência, embora apenas para problemas convexos
- A taxa de aprendizado no RMSProp é fixa ao longo do tempo, sendo mais adequado para tarefas de treinamento de longa duração

RMSProp × ADAGRAD

- O RMSProp é uma abordagem heurística, enquanto o ADAGRAD possui limites formais na sua taxa de convergência, embora apenas para problemas convexos
- A taxa de aprendizado no RMSProp é fixa ao longo do tempo, sendo mais adequado para tarefas de treinamento de longa duração
- A magnitude da soma dos gradientes quadráticos do ADAGRAD cresce linearmente com o tempo t ; portanto, a taxa de aprendizado no ADAGRAD reduz em $1/\sqrt{t}$, o que é bastante agressivo (podendo ser bom para modelos curtos e fáceis de treinar, porém rápido demais para treinos de longa duração)

RMSProp × ADAGRAD



Crédito: Alec Radford, 2015

ADAM – *Adaptive Moment Estimation*

[Kingma and Ba, 2014]

(incompleto, mas bem próximo)

```
# Adam
m = beta1*m + (1-beta1)*dx # update first moment
v = beta2*v + (1-beta2)*(dx**2) # update second moment
x += - learning_rate * m / (np.sqrt(v) + 1e-7)
```

ADAM – Adaptive Moment Estimation

[Kingma and Ba, 2014]

(incompleto, mas bem próximo)

```
# Adam
m = beta1*m + (1-beta1)*dx # update first moment
v = beta2*v + (1-beta2)*(dx**2) # update second moment
x += - learning_rate * m / (np.sqrt(v) + 1e-7)
```

momentum

RMSProp-like

ADAM – Adaptive Moment Estimation

[Kingma and Ba, 2014]

(incompleto, mas bem próximo)

```
# Adam
m = beta1*m + (1-beta1)*dx # update first moment
v = beta2*v + (1-beta2)*(dx**2) # update second moment
x += - learning_rate * m / (np.sqrt(v) + 1e-7)
```

momentum

RMSProp-like

Semelhante ao RMSProp com “*momentum*”

```
# RMSProp
cache = decay_rate * cache + (1 - decay_rate) * dx**2
x += - learning_rate * dx / (np.sqrt(cache) + 1e-7)
```

ADAM – *Adaptive Moment Estimation*

[Kingma and Ba, 2014]

```
# Adam
m,v = #... initialize caches to zeros
for t in xrange(1, big_number):
    dx = # ... evaluate gradient
    m = beta1*m + (1-beta1)*dx # update first moment
    v = beta2*v + (1-beta2)*(dx**2) # update second moment
    mb = m/(1-beta1**t) # correct bias
    vb = v/(1-beta2**t) # correct bias
    x += - learning_rate * mb / (np.sqrt(vb) + 1e-7)
```

ADAM – Adaptive Moment Estimation

[Kingma and Ba, 2014]

```
# Adam
m,v = #... initialize caches to zeros
for t in xrange(1, big_number):
    dx = # ... evaluate gradient
    m = beta1*m + (1-beta1)*dx # update first moment
    v = beta2*v + (1-beta2)*(dx**2) # update second moment
    mb = m/(1-beta1**t) # correct bias
    vb = v/(1-beta2**t) # correct bias
    x += - learning_rate * mb / (np.sqrt(vb) + 1e-7)
```

momentum

ADAM – Adaptive Moment Estimation

[Kingma and Ba, 2014]

```
# Adam
m,v = #... initialize caches to zeros
for t in xrange(1, big_number):
    dx = # ... evaluate gradient
    m = beta1*m + (1-beta1)*dx # update first moment
    v = beta2*v + (1-beta2)*(dx**2) # update second moment
    mb = m/(1-beta1**t) # correct bias
    vb = v/(1-beta2**t) # correct bias
    x += - learning_rate * mb / (np.sqrt(vb) + 1e-7)
```

momentum

RMSProp-like (1)

RMSProp-like (2)

ADAM – Adaptive Moment Estimation

[Kingma and Ba, 2014]

```
# Adam
m,v = #... initialize caches to zeros
for t in xrange(1, big_number):
    dx = # ... evaluate gradient
    m = beta1*m + (1-beta1)*dx # update first moment
    v = beta2*v + (1-beta2)*(dx**2) # update second moment
    mb = m/(1-beta1**t) # correct bias
    vb = v/(1-beta2**t) # correct bias
    x += - learning_rate * mb / (np.sqrt(vb) + 1e-7)
```

momentum

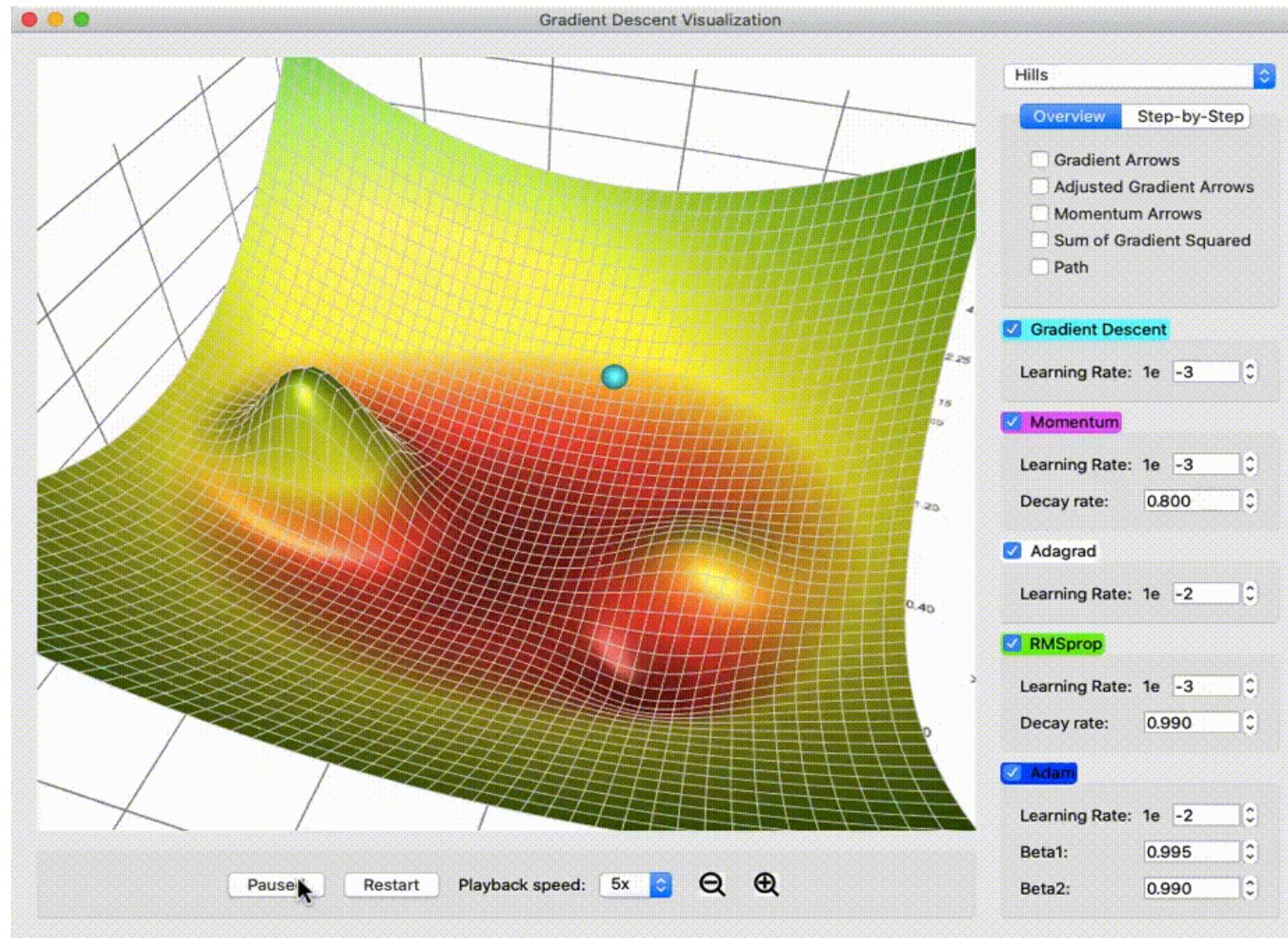
RMSProp-like (1)

Correção de viés
(só relevante nas primeiras
iterações quando t é pequeno)

RMSProp-like (2)

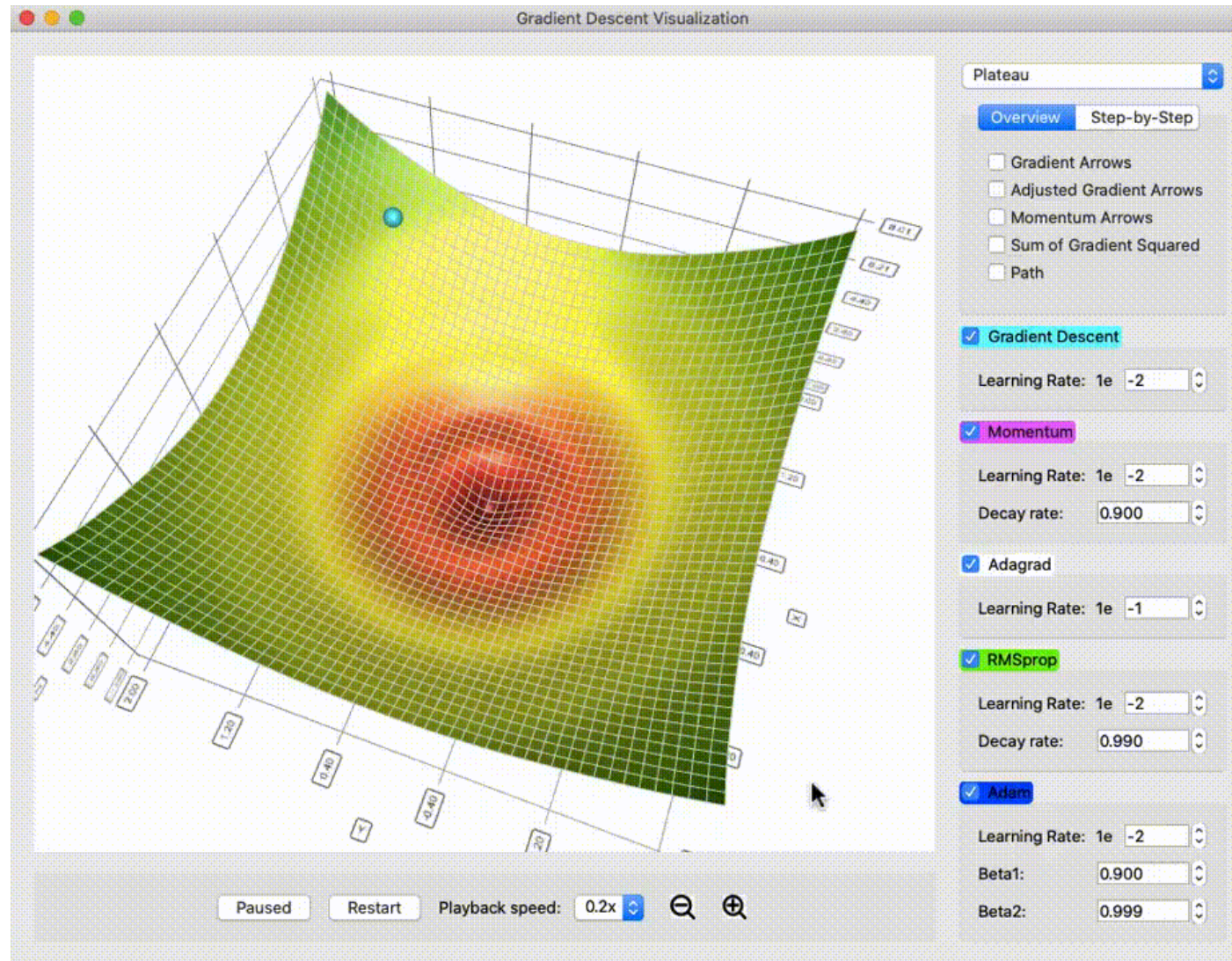
A correção de viés compensa o fato de que m e v são inicializados em zero e precisam de algum tempo para “aquecer”, isto é, obter valores adequados

Vários Esquemas de Atualização de Pesos



Crédito: Lili Jiang, 2020

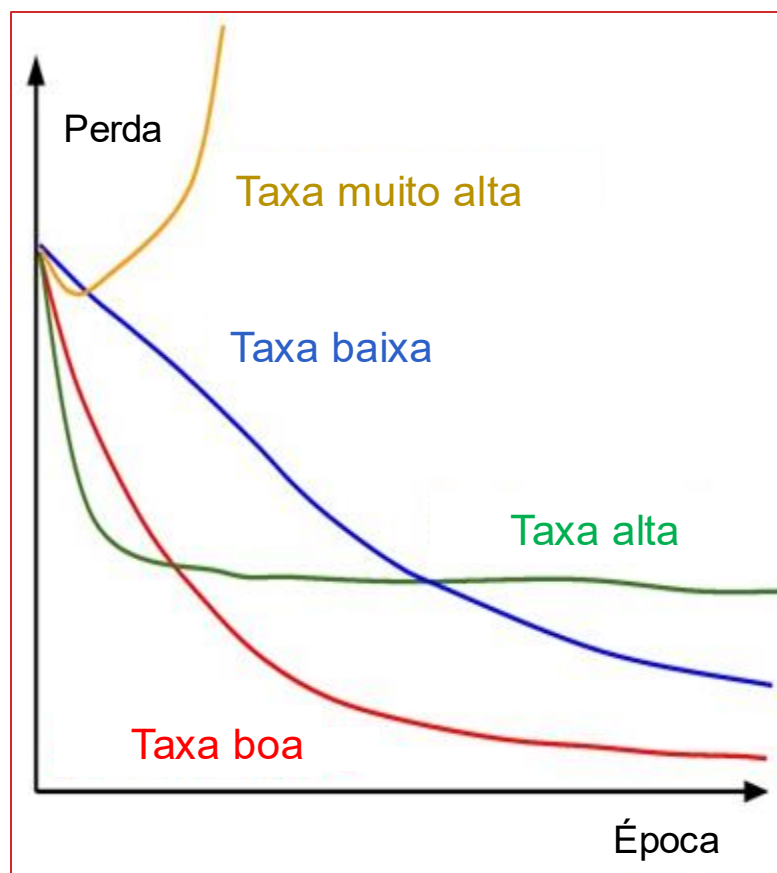
Vários Esquemas de Atualização de Pesos



Crédito: Lili Jiang, 2020

Atualização × Taxa de Aprendizado

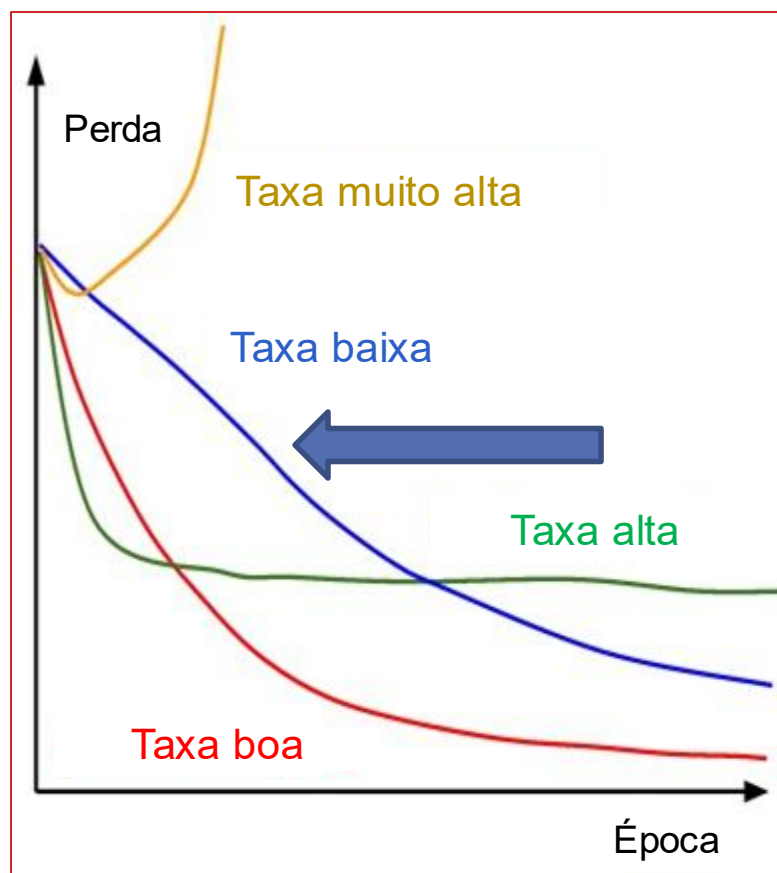
SGD, SGD + *Momentum*, SGD + NAG, ADAGRAD, RMSProp e ADAM possuem a **taxa de aprendizado** como um hiperparâmetro



P: Qual dessas é a melhor taxa de aprendizado a se utilizar?

Atualização × Taxa de Aprendizado

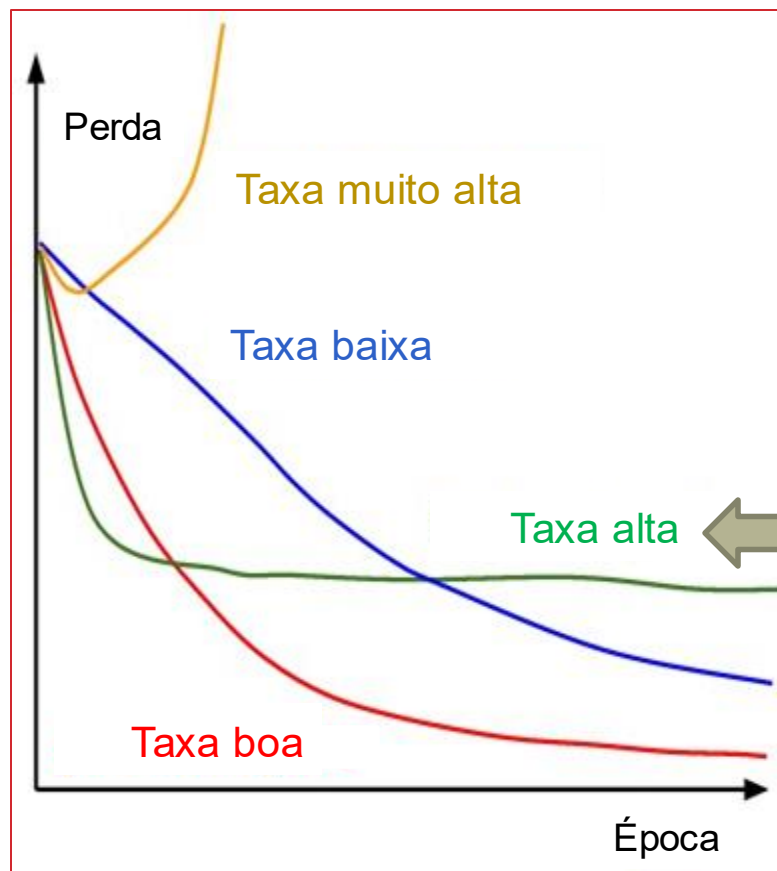
SGD, SGD + *Momentum*, SGD + NAG, ADAGRAD, RMSProp e ADAM possuem a **taxa de aprendizado** como um hiperparâmetro



P: Qual dessas é a melhor taxa de aprendizado a se utilizar?

Atualização × Taxa de Aprendizado

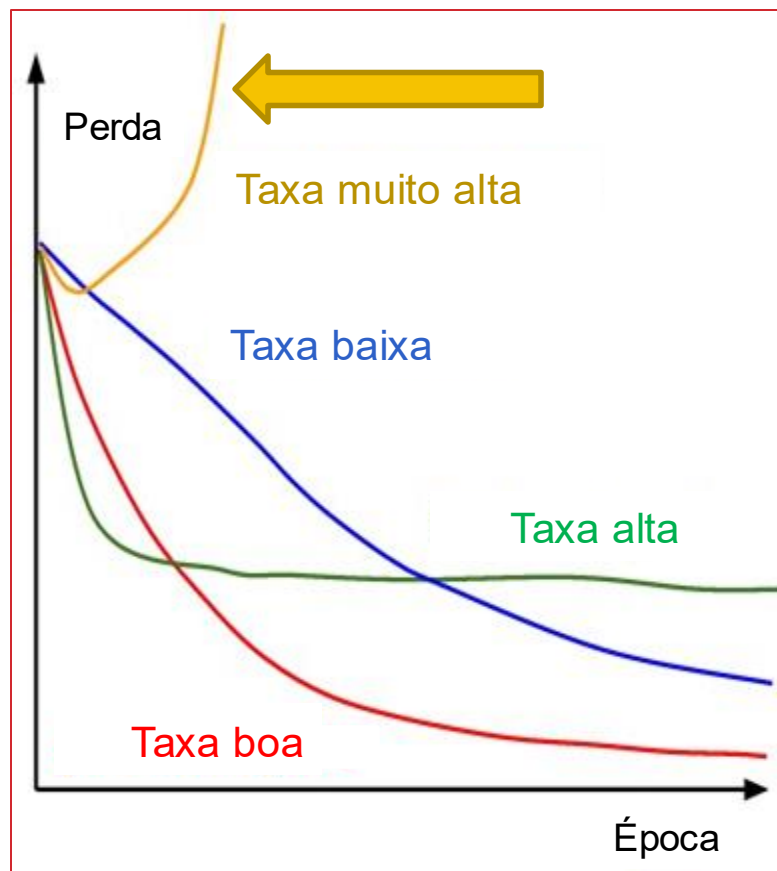
SGD, SGD + *Momentum*, SGD + NAG, ADAGRAD, RMSProp e ADAM possuem a **taxa de aprendizado** como um hiperparâmetro



P: Qual dessas é a melhor taxa de aprendizado a se utilizar?

Atualização × Taxa de Aprendizado

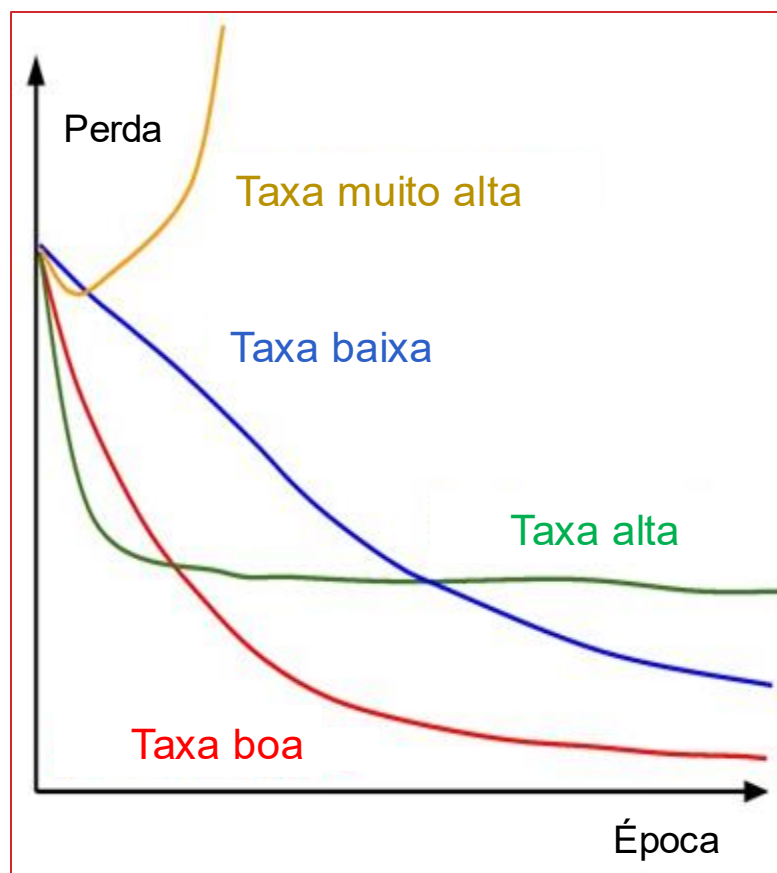
SGD, SGD + *Momentum*, SGD + NAG, ADAGRAD, RMSProp e ADAM possuem a **taxa de aprendizado** como um hiperparâmetro



P: Qual dessas é a melhor taxa de aprendizado a se utilizar?

Atualização × Taxa de Aprendizado

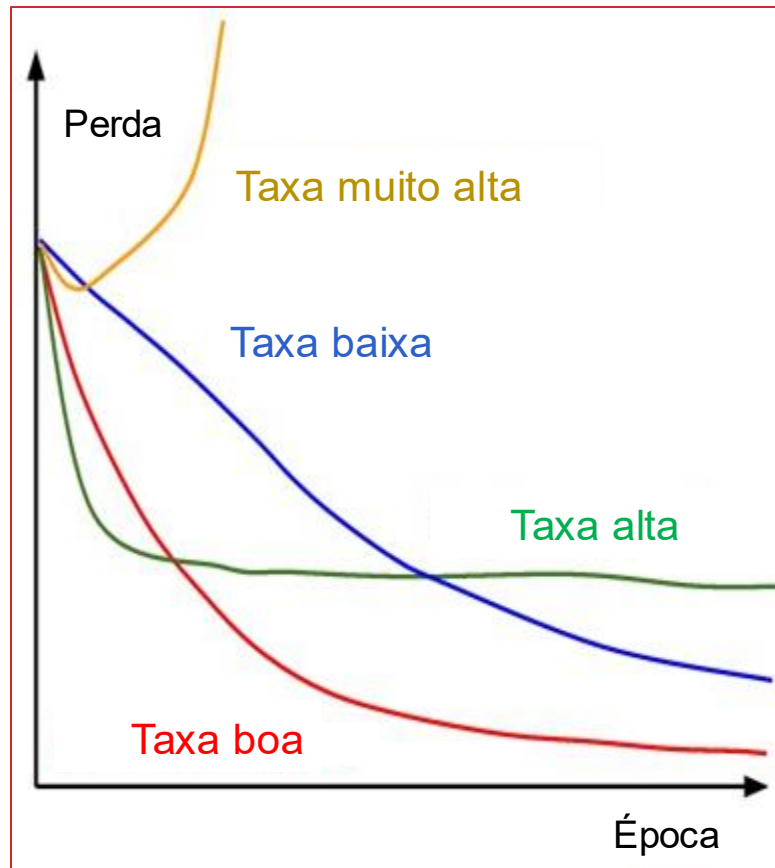
SGD, SGD + *Momentum*, SGD + NAG, ADAGRAD, RMSProp e ADAM possuem a **taxa de aprendizado** como um hiperparâmetro



P: Qual dessas é a melhor taxa de aprendizado a se utilizar?

Decaimento da Taxa de Aprendizado

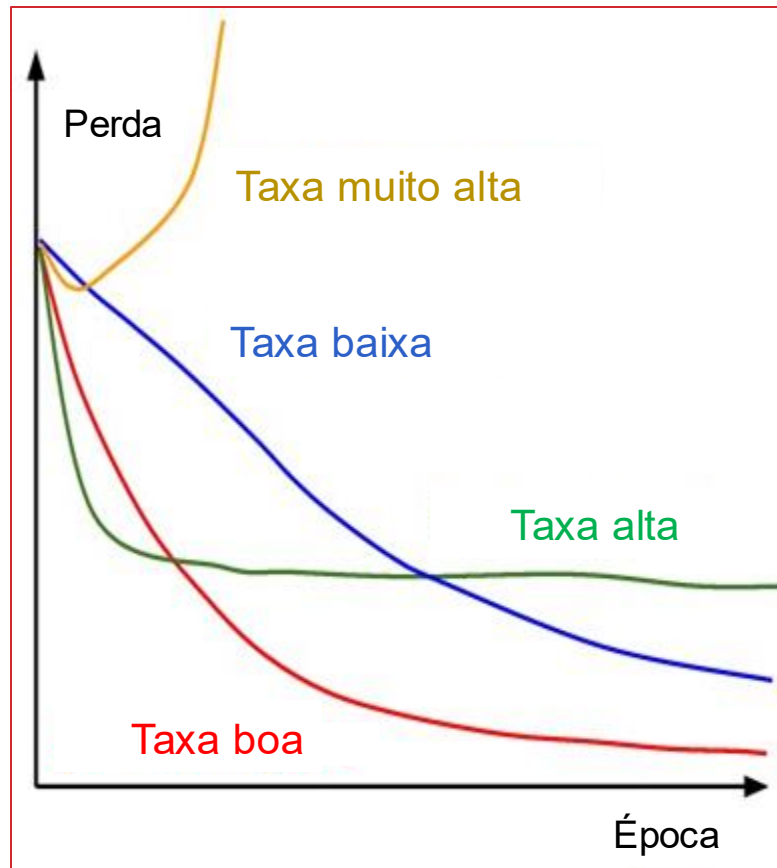
SGD, SGD + *Momentum*, SGD + NAG, ADAGRAD, RMSProp e ADAM possuem a **taxa de aprendizado** como um hiperparâmetro



⇒ **Decaimento da taxa de aprendizado ao longo do tempo!**

Decaimento da Taxa de Aprendizizado

SGD, SGD + *Momentum*, SGD + NAG, ADAGRAD, RMSProp e ADAM possuem a **taxa de aprendizado** como um hiperparâmetro



⇒ **Decaimento da taxa de aprendizado ao longo do tempo!**

- **Constante:** p.ex., dividir por 2 a intervalos fixos
- **Exponencial:** $\alpha = \alpha_0 e^{-kt}$
- **1/t:** $\alpha = \alpha_0 / (1 + kt)$
- **1/√t:** $\alpha = \alpha_0 / \sqrt{1 + kt}$ (p.ex., ADAGRAD)

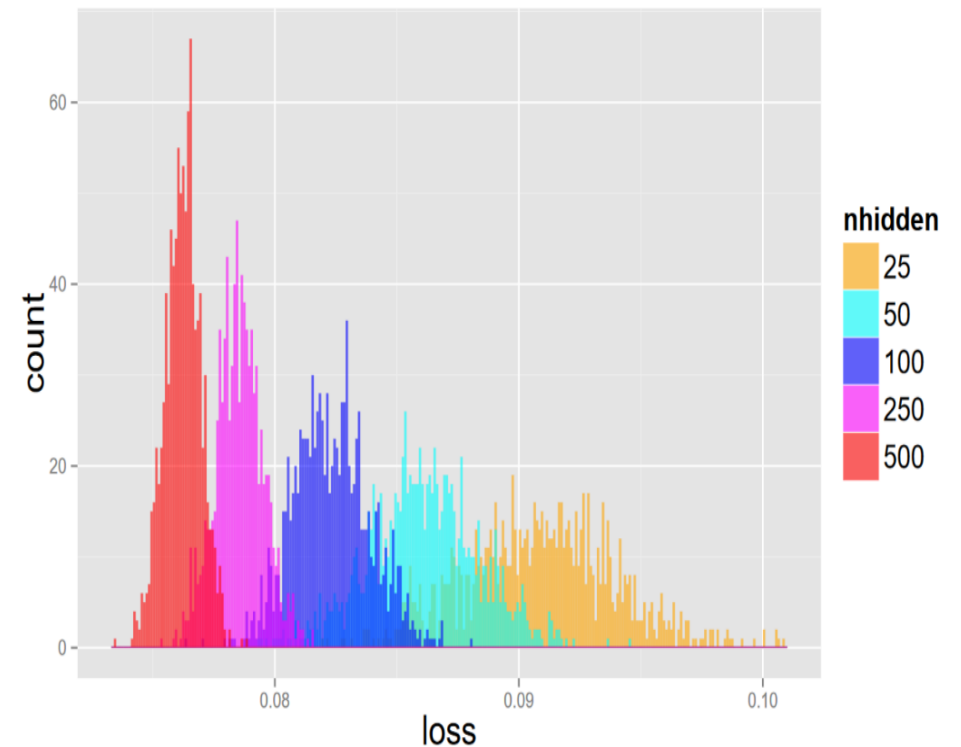
Múltiplos Mínimos Locais são um Problema?

Elas acontecem, mas a maioria dos mínimos locais tem perda semelhante ao mínimo global

Múltiplos Mínimos Locais são um Problema?

Elas acontecem, mas a maioria dos mínimos locais tem perda semelhante ao mínimo global

Para uma simples rede de imagens, na medida que a rede fica mais complexa (mais unidades ocultas), existem mais mínimos locais agrupados próximos da perda global mínima



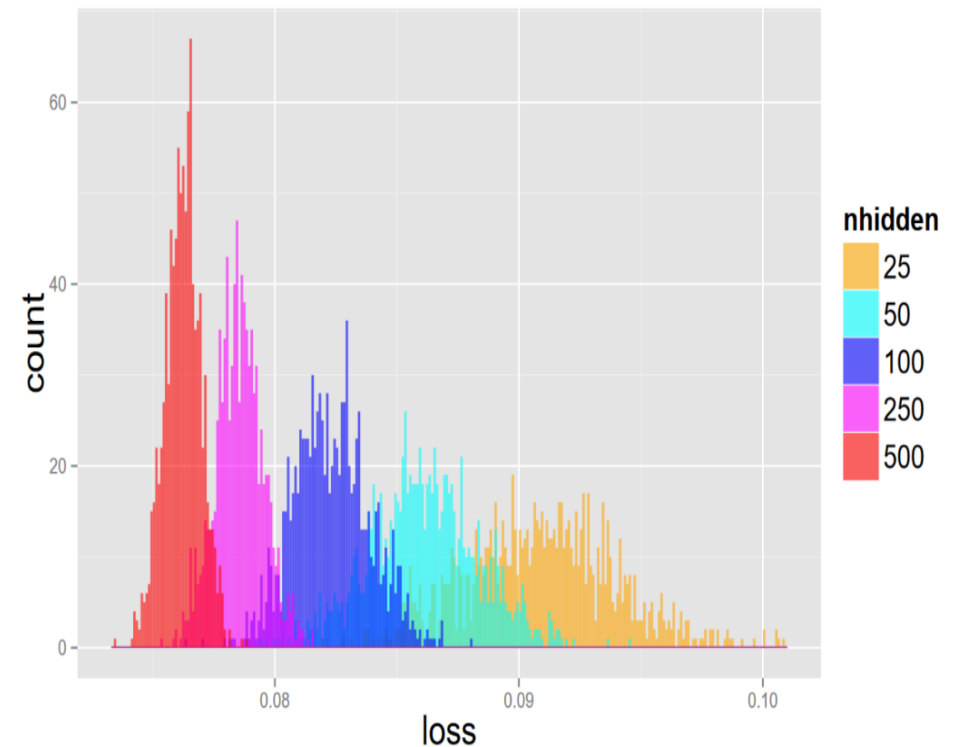
Veja Choromanska, Henaff, Mathieu, Ben Arous and Lecun "The Loss Surface of Multilayer Networks" arXiv 1412.0233, 2014

Múltiplos Mínimos Locais são um Problema?

Elas acontecem, mas a maioria dos mínimos locais tem perda semelhante ao mínimo global

Para uma simples rede de imagens, na medida que a rede fica mais complexa (mais unidades ocultas), existem mais mínimos locais agrupados próximos da perda global mínima

Portanto, é bom, e até mesmo desejável, projetar redes muito complexas para aliviar o problema dos mínimos locais



Veja Choromanska, Henaff, Mathieu, Ben Arous and Lecun "The Loss Surface of Multilayer Networks" arXiv 1412.0233, 2014

Normalização em Lote (*Batch Normalization*)

“Você quer ativações gaussianas unitárias? Apenas faça-as assim.”

[Ioffe & Szegedy, 2015]

Normalização em Lote (*Batch Normalization*)

“Você quer ativações gaussianas unitárias? Apenas faça-as assim.”

[Ioffe & Szegedy, 2015]

Considere um lote de ativações em uma camada qualquer.

Para fazer com que cada dimensão se comporte como gaussiana unitária, aplica-se:

$$\hat{x}^{(k)} = \frac{x^{(k)} - \mathbb{E}[x^{(k)}]}{\sqrt{\text{Var}[x^{(k)}]}}$$

Normalização em Lote (*Batch Normalization*)

“Você quer ativações gaussianas unitárias? Apenas faça-as assim.”

[Ioffe & Szegedy, 2015]

Considere um lote de ativações em uma camada qualquer.

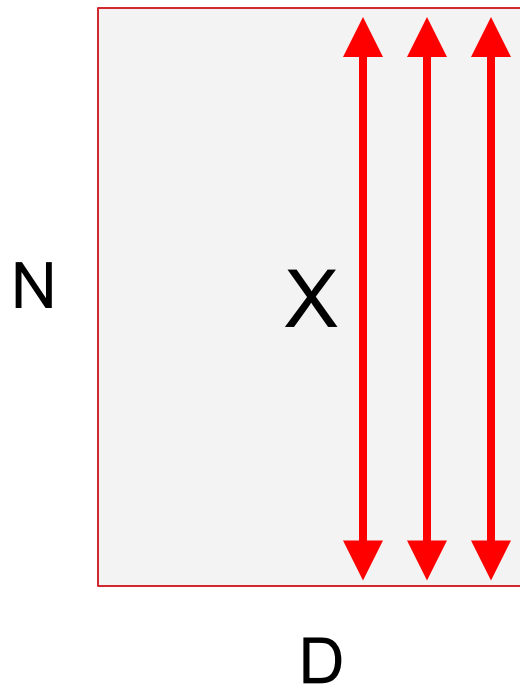
Para fazer com que cada dimensão se comporte como gaussiana unitária, aplica-se:

$$\hat{x}^{(k)} = \frac{x^{(k)} - \mathbb{E}[x^{(k)}]}{\sqrt{\text{Var}[x^{(k)}]}} \longrightarrow \text{Esta é uma função diferenciável comum ...}$$

Normalização em Lote (*Batch Normalization*)

“Você quer ativações gaussianas unitárias? Apenas faça-as assim.”

[Ioffe & Szegedy, 2015]

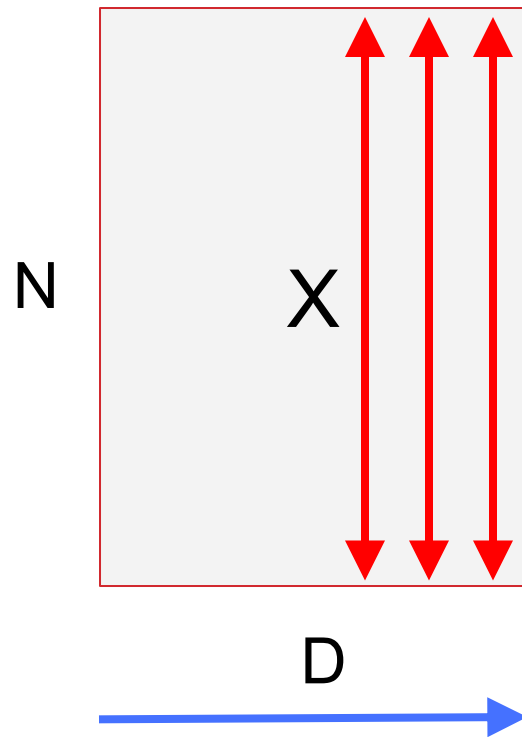


1. Calcular a média e a variância empírica para cada dimensão de forma independente

Normalização em Lote (*Batch Normalization*)

“Você quer ativações gaussianas unitárias? Apenas faça-as assim.”

[Ioffe & Szegedy, 2015]

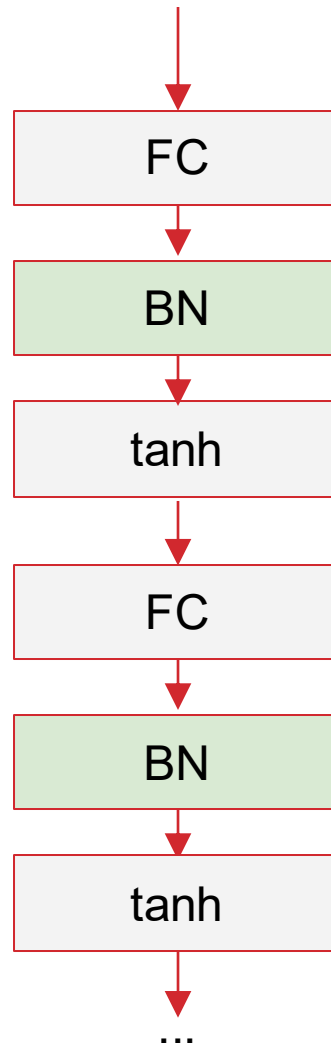


1. Calcular a média e a variância empírica para cada dimensão de forma independente

2. Normalizar

$$\hat{x}^{(k)} = \frac{x^{(k)} - E[x^{(k)}]}{\sqrt{\text{Var}[x^{(k)}]}}$$

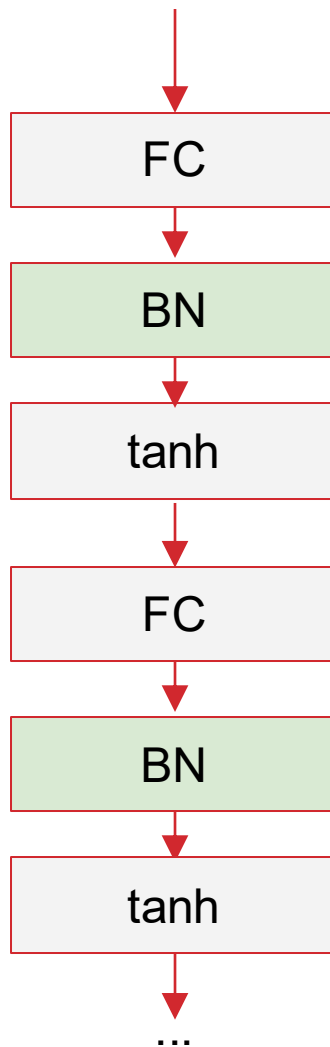
Normalização em Lote (*Batch Normalization*)



Normalmente inserida após as camadas completamente conectadas (ou camadas convolucionais), mas antes da não linearidade.

$$\hat{x}^{(k)} = \frac{x^{(k)} - E[x^{(k)}]}{\sqrt{\text{Var}[x^{(k)}]}}$$

Normalização em Lote (*Batch Normalization*)



Normalmente inserida após as camadas completamente conectadas (ou camadas convolucionais), mas antes da não linearidade.

Problema: deseja-se realmente uma entrada gaussiana unitária para a não-linearidade?

$$\hat{x}^{(k)} = \frac{x^{(k)} - E[x^{(k)}]}{\sqrt{\text{Var}[x^{(k)}]}}$$

Normalização em Lote (*Batch Normalization*)

Solução!

Normalizar:

$$\hat{x}^{(k)} = \frac{x^{(k)} - \mathbb{E}[x^{(k)}]}{\sqrt{\text{Var}[x^{(k)}]}}$$

Normalização em Lote (*Batch Normalization*)

Solução!

Normalizar:

$$\hat{x}^{(k)} = \frac{x^{(k)} - \mathbb{E}[x^{(k)}]}{\sqrt{\text{Var}[x^{(k)}]}}$$

E, em seguida, permitir que a rede ajuste a saída para outro intervalo, se quiser:

$$y^{(k)} = \gamma^{(k)} \hat{x}^{(k)} + \beta^{(k)}$$

em que $\gamma^{(k)}$ e $\beta^{(k)}$ devem ser aprendidos pela rede

Normalização em Lote (*Batch Normalization*)

Solução!

Normalizar:

$$\hat{x}^{(k)} = \frac{x^{(k)} - E[x^{(k)}]}{\sqrt{\text{Var}[x^{(k)}]}}$$

E, em seguida, permitir que a rede ajuste a saída para outro intervalo, se quiser:

$$y^{(k)} = \gamma^{(k)} \hat{x}^{(k)} + \beta^{(k)}$$

em que $\gamma^{(k)}$ e $\beta^{(k)}$ devem ser aprendidos pela rede

Observe que a rede poderia aprender, de modo que:

$$\gamma^{(k)} = \sqrt{\text{Var}[x^{(k)}]}$$

$$\beta^{(k)} = E[x^{(k)}]$$

e, assim, recuperar o mapeamento de identidade

Normalização em Lote (*Batch Normalization*)

$$\frac{\partial \ell}{\partial \hat{x}_i} = \frac{\partial \ell}{\partial y_i} \cdot \gamma$$

$$\frac{\partial \ell}{\partial \sigma_{\mathcal{B}}^2} = \sum_{i=1}^m \frac{\partial \ell}{\partial \hat{x}_i} \cdot (x_i - \mu_{\mathcal{B}}) \cdot \frac{-1}{2} (\sigma_{\mathcal{B}}^2 + \epsilon)^{-3/2}$$

$$\frac{\partial \ell}{\partial \mu_{\mathcal{B}}} = \left(\sum_{i=1}^m \frac{\partial \ell}{\partial \hat{x}_i} \cdot \frac{-1}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}} \right) + \frac{\partial \ell}{\partial \sigma_{\mathcal{B}}^2} \cdot \frac{\sum_{i=1}^m -2(x_i - \mu_{\mathcal{B}})}{m}$$

$$\frac{\partial \ell}{\partial x_i} = \frac{\partial \ell}{\partial \hat{x}_i} \cdot \frac{1}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}} + \frac{\partial \ell}{\partial \sigma_{\mathcal{B}}^2} \cdot \frac{2(x_i - \mu_{\mathcal{B}})}{m} + \frac{\partial \ell}{\partial \mu_{\mathcal{B}}} \cdot \frac{1}{m}$$

$$\frac{\partial \ell}{\partial \gamma} = \sum_{i=1}^m \frac{\partial \ell}{\partial y_i} \cdot \hat{x}_i$$

$$\frac{\partial \ell}{\partial \beta} = \sum_{i=1}^m \frac{\partial \ell}{\partial y_i}$$

Não necessários diretamente, são subexpressões para os outros gradientes.

Considerar como “backprop” para $\hat{x}$, $\sigma_{\mathcal{B}}^2$, $\mu_{\mathcal{B}}$, que são internos a atualização do minibatch.

Normalização em Lote (*Batch Normalization*)

$$\frac{\partial \ell}{\partial \hat{x}_i} = \frac{\partial \ell}{\partial y_i} \cdot \gamma$$

$$\frac{\partial \ell}{\partial \sigma_{\mathcal{B}}^2} = \sum_{i=1}^m \frac{\partial \ell}{\partial \hat{x}_i} \cdot (x_i - \mu_{\mathcal{B}}) \cdot \frac{-1}{2} (\sigma_{\mathcal{B}}^2 + \epsilon)^{-3/2}$$

$$\frac{\partial \ell}{\partial \mu_{\mathcal{B}}} = \left(\sum_{i=1}^m \frac{\partial \ell}{\partial \hat{x}_i} \cdot \frac{-1}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}} \right) + \frac{\partial \ell}{\partial \sigma_{\mathcal{B}}^2} \cdot \frac{\sum_{i=1}^m -2(x_i - \mu_{\mathcal{B}})}{m}$$

$$\frac{\partial \ell}{\partial x_i} = \frac{\partial \ell}{\partial \hat{x}_i} \cdot \frac{1}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}} + \frac{\partial \ell}{\partial \sigma_{\mathcal{B}}^2} \cdot \frac{2(x_i - \mu_{\mathcal{B}})}{m} + \frac{\partial \ell}{\partial \mu_{\mathcal{B}}} \cdot \frac{1}{m}$$

Gradientes para propagar para a camada de entrada

$$\frac{\partial \ell}{\partial \gamma} = \sum_{i=1}^m \frac{\partial \ell}{\partial y_i} \cdot \hat{x}_i$$

$$\frac{\partial \ell}{\partial \beta} = \sum_{i=1}^m \frac{\partial \ell}{\partial y_i}$$

Normalização em Lote (*Batch Normalization*)

$$\frac{\partial \ell}{\partial \hat{x}_i} = \frac{\partial \ell}{\partial y_i} \cdot \gamma$$

$$\frac{\partial \ell}{\partial \sigma_{\mathcal{B}}^2} = \sum_{i=1}^m \frac{\partial \ell}{\partial \hat{x}_i} \cdot (x_i - \mu_{\mathcal{B}}) \cdot \frac{-1}{2} (\sigma_{\mathcal{B}}^2 + \epsilon)^{-3/2}$$

$$\frac{\partial \ell}{\partial \mu_{\mathcal{B}}} = \left(\sum_{i=1}^m \frac{\partial \ell}{\partial \hat{x}_i} \cdot \frac{-1}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}} \right) + \frac{\partial \ell}{\partial \sigma_{\mathcal{B}}^2} \cdot \frac{\sum_{i=1}^m -2(x_i - \mu_{\mathcal{B}})}{m}$$

$$\frac{\partial \ell}{\partial x_i} = \frac{\partial \ell}{\partial \hat{x}_i} \cdot \frac{1}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}} + \frac{\partial \ell}{\partial \sigma_{\mathcal{B}}^2} \cdot \frac{2(x_i - \mu_{\mathcal{B}})}{m} + \frac{\partial \ell}{\partial \mu_{\mathcal{B}}} \cdot \frac{1}{m}$$

$$\frac{\partial \ell}{\partial \gamma} = \sum_{i=1}^m \frac{\partial \ell}{\partial y_i} \cdot \hat{x}_i$$

$$\frac{\partial \ell}{\partial \beta} = \sum_{i=1}^m \frac{\partial \ell}{\partial y_i}$$

Gradientes para os parâmetros γ e β .

Normalização em Lote (*Batch Normalization*)

Input: Values of x over a mini-batch: $\mathcal{B} = \{x_{1\dots m}\}$;

Parameters to be learned: γ, β

Output: $\{y_i = \text{BN}_{\gamma, \beta}(x_i)\}$

$$\mu_{\mathcal{B}} \leftarrow \frac{1}{m} \sum_{i=1}^m x_i \quad // \text{ mini-batch mean}$$

$$\sigma_{\mathcal{B}}^2 \leftarrow \frac{1}{m} \sum_{i=1}^m (x_i - \mu_{\mathcal{B}})^2 \quad // \text{ mini-batch variance}$$

$$\hat{x}_i \leftarrow \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}} \quad // \text{ normalize}$$

$$y_i \leftarrow \gamma \hat{x}_i + \beta \equiv \text{BN}_{\gamma, \beta}(x_i) \quad // \text{ scale and shift}$$

- Melhora o fluxo gradiente através da rede

Normalização em Lote (*Batch Normalization*)

Input: Values of x over a mini-batch: $\mathcal{B} = \{x_{1\dots m}\}$;

Parameters to be learned: γ, β

Output: $\{y_i = \text{BN}_{\gamma, \beta}(x_i)\}$

$$\mu_{\mathcal{B}} \leftarrow \frac{1}{m} \sum_{i=1}^m x_i \quad // \text{ mini-batch mean}$$

$$\sigma_{\mathcal{B}}^2 \leftarrow \frac{1}{m} \sum_{i=1}^m (x_i - \mu_{\mathcal{B}})^2 \quad // \text{ mini-batch variance}$$

$$\hat{x}_i \leftarrow \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}} \quad // \text{ normalize}$$

$$y_i \leftarrow \gamma \hat{x}_i + \beta \equiv \text{BN}_{\gamma, \beta}(x_i) \quad // \text{ scale and shift}$$

- Melhora o fluxo gradiente através da rede
- Permite taxas de aprendizagem mais altas

Normalização em Lote (*Batch Normalization*)

Input: Values of x over a mini-batch: $\mathcal{B} = \{x_{1\dots m}\}$;

Parameters to be learned: γ, β

Output: $\{y_i = \text{BN}_{\gamma, \beta}(x_i)\}$

$$\mu_{\mathcal{B}} \leftarrow \frac{1}{m} \sum_{i=1}^m x_i \quad // \text{ mini-batch mean}$$

$$\sigma_{\mathcal{B}}^2 \leftarrow \frac{1}{m} \sum_{i=1}^m (x_i - \mu_{\mathcal{B}})^2 \quad // \text{ mini-batch variance}$$

$$\hat{x}_i \leftarrow \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}} \quad // \text{ normalize}$$

$$y_i \leftarrow \gamma \hat{x}_i + \beta \equiv \text{BN}_{\gamma, \beta}(x_i) \quad // \text{ scale and shift}$$

- Melhora o fluxo gradiente através da rede
- Permite taxas de aprendizagem mais altas
- Reduz a forte dependência da inicialização

Normalização em Lote (*Batch Normalization*)

Input: Values of x over a mini-batch: $\mathcal{B} = \{x_{1\dots m}\}$;

Parameters to be learned: γ, β

Output: $\{y_i = \text{BN}_{\gamma, \beta}(x_i)\}$

$$\mu_{\mathcal{B}} \leftarrow \frac{1}{m} \sum_{i=1}^m x_i \quad // \text{ mini-batch mean}$$

$$\sigma_{\mathcal{B}}^2 \leftarrow \frac{1}{m} \sum_{i=1}^m (x_i - \mu_{\mathcal{B}})^2 \quad // \text{ mini-batch variance}$$

$$\hat{x}_i \leftarrow \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}} \quad // \text{ normalize}$$

$$y_i \leftarrow \gamma \hat{x}_i + \beta \equiv \text{BN}_{\gamma, \beta}(x_i) \quad // \text{ scale and shift}$$

- Melhora o fluxo gradiente através da rede
- Permite taxas de aprendizagem mais altas
- Reduz a forte dependência da inicialização
- Atua como uma forma de regularização diferente, reduzindo talvez a necessidade de *dropout*

Normalização em Lote (*Batch Normalization*)

Input: Values of x over a mini-batch: $\mathcal{B} = \{x_{1\dots m}\}$;

Parameters to be learned: γ, β

Output: $\{y_i = \text{BN}_{\gamma, \beta}(x_i)\}$

$$\mu_{\mathcal{B}} \leftarrow \frac{1}{m} \sum_{i=1}^m x_i \quad // \text{ mini-batch mean}$$

$$\sigma_{\mathcal{B}}^2 \leftarrow \frac{1}{m} \sum_{i=1}^m (x_i - \mu_{\mathcal{B}})^2 \quad // \text{ mini-batch variance}$$

$$\hat{x}_i \leftarrow \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}} \quad // \text{ normalize}$$

$$y_i \leftarrow \gamma \hat{x}_i + \beta \equiv \text{BN}_{\gamma, \beta}(x_i) \quad // \text{ scale and shift}$$

- Melhora o fluxo gradiente através da rede
- Permite taxas de aprendizagem mais altas
- Reduz a forte dependência da inicialização
- Atua como uma forma de regularização diferente, reduzindo talvez a necessidade de *dropout*

Desnormalização!!

Aprende-se γ e β (mesmas dims que μ e σ^2)

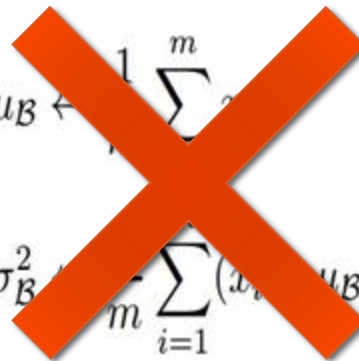
Permite (se necessário) aprender o mapeamento identidade!

Normalização em Lote (*Batch Normalization*)

Input: Values of x over a mini-batch: $\mathcal{B} = \{x_{1\dots m}\}$;

Parameters to be learned: γ, β

Output: $\{y_i = \text{BN}_{\gamma, \beta}(x_i)\}$


$$\mu_{\mathcal{B}} \leftarrow \frac{1}{m} \sum_{i=1}^m x_i \quad // \text{ mini-batch mean}$$

$$\sigma_{\mathcal{B}}^2 \leftarrow \frac{1}{m} \sum_{i=1}^m (x_i - \mu_{\mathcal{B}})^2 \quad // \text{ mini-batch variance}$$

$$\hat{x}_i \leftarrow \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}} \quad // \text{ normalize}$$

$$y_i \leftarrow \gamma \hat{x}_i + \beta \equiv \text{BN}_{\gamma, \beta}(x_i) \quad // \text{ scale and shift}$$

Nota: no momento do **teste**, a camada de normalização em lote funciona diferente:

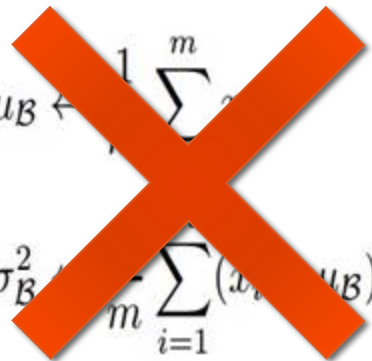
- A **média e a variância não são calculadas** com base no lote

Normalização em Lote (*Batch Normalization*)

Input: Values of x over a mini-batch: $\mathcal{B} = \{x_{1\dots m}\}$;

Parameters to be learned: γ, β

Output: $\{y_i = \text{BN}_{\gamma, \beta}(x_i)\}$


$$\mu_{\mathcal{B}} \leftarrow \frac{1}{m} \sum_{i=1}^m x_i \quad // \text{ mini-batch mean}$$

$$\sigma_{\mathcal{B}}^2 \leftarrow \frac{1}{m} \sum_{i=1}^m (x_i - \mu_{\mathcal{B}})^2 \quad // \text{ mini-batch variance}$$

$$\hat{x}_i \leftarrow \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}} \quad // \text{ normalize}$$

$$y_i \leftarrow \gamma \hat{x}_i + \beta \equiv \text{BN}_{\gamma, \beta}(x_i) \quad // \text{ scale and shift}$$

Nota: no momento do **teste**, a camada de normalização em lote funciona diferente:

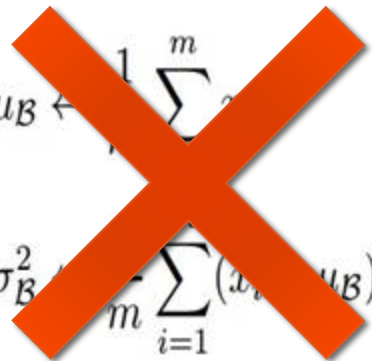
- A **média e a variância não são calculadas** com base no lote
- Em vez disso, **utiliza-se um único par fixo de média e variância** empírica de ativações obtido durante o treinamento

Normalização em Lote (*Batch Normalization*)

Input: Values of x over a mini-batch: $\mathcal{B} = \{x_{1...m}\}$;

Parameters to be learned: γ, β

Output: $\{y_i = \text{BN}_{\gamma, \beta}(x_i)\}$


$$\mu_{\mathcal{B}} \leftarrow \frac{1}{m} \sum_{i=1}^m x_i \quad // \text{ mini-batch mean}$$

$$\sigma_{\mathcal{B}}^2 \leftarrow \frac{1}{m} \sum_{i=1}^m (x_i - \mu_{\mathcal{B}})^2 \quad // \text{ mini-batch variance}$$

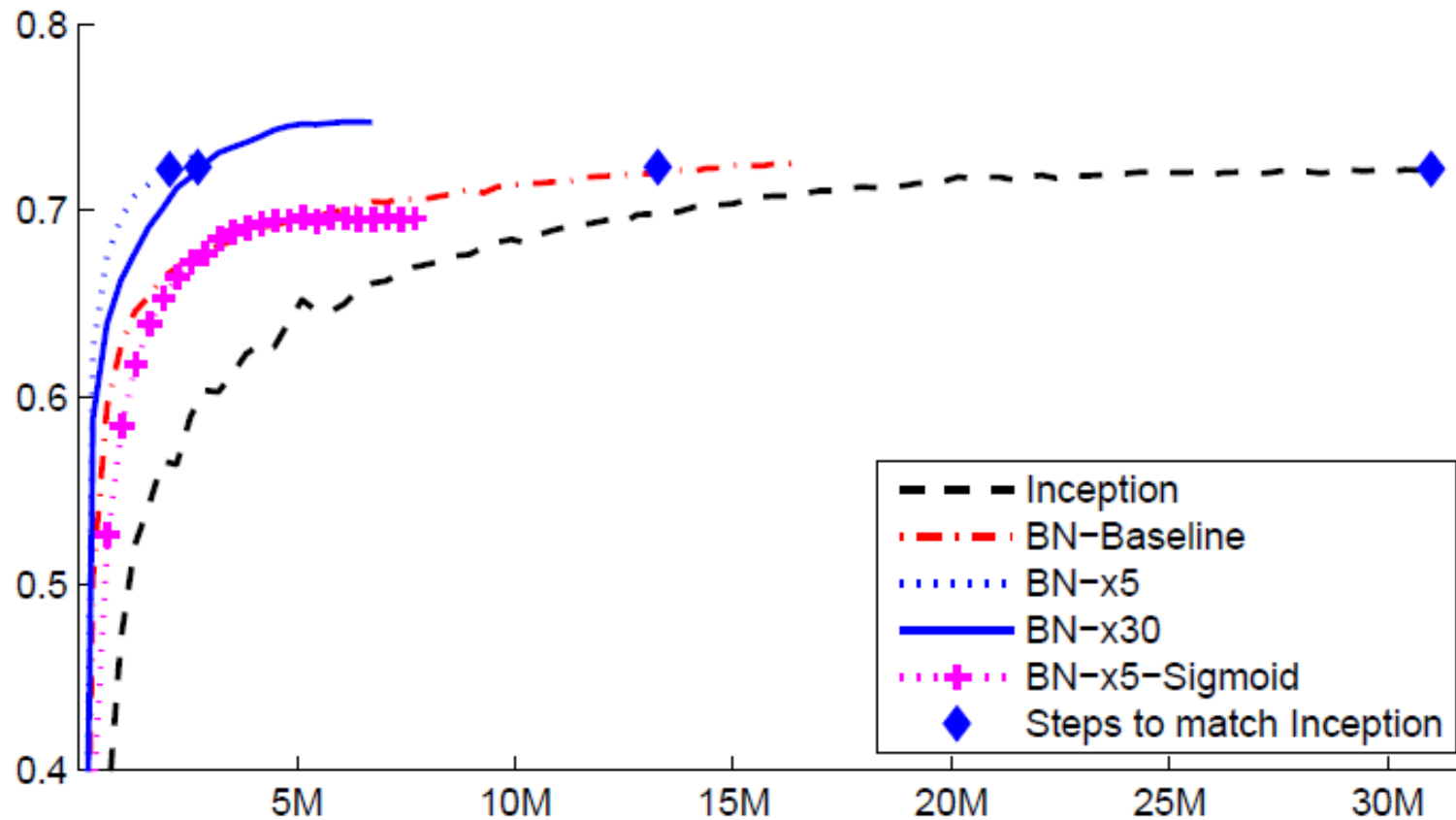
$$\hat{x}_i \leftarrow \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}} \quad // \text{ normalize}$$

$$y_i \leftarrow \gamma \hat{x}_i + \beta \equiv \text{BN}_{\gamma, \beta}(x_i) \quad // \text{ scale and shift}$$

Nota: no momento do **teste**, a camada de normalização em lote funciona diferente:

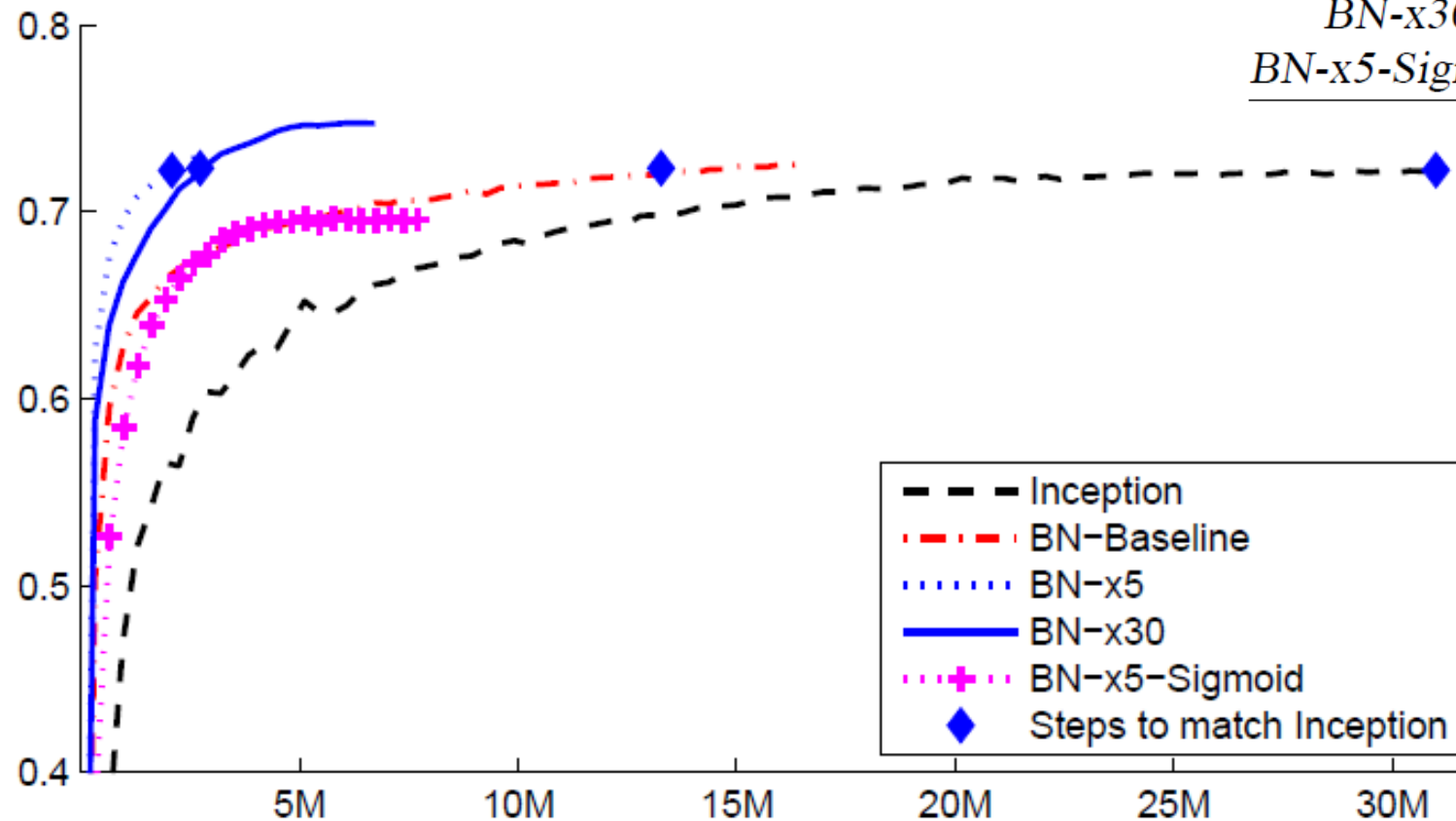
- A **média e a variância não são calculadas** com base no lote
- Em vez disso, **utiliza-se um único par fixo de média e variância** empírica de ativações obtido durante o treinamento
- Por exemplo, pode-se estimar o par durante o treinamento por meio de médias móveis

Normalização em Lote (*Batch Normalization*)



Normalização em Lote (*Batch Normalization*)

Model	Steps to 72.2%	Max accuracy
Inception	$31.0 \cdot 10^6$	72.2%
<i>BN-Baseline</i>	$13.3 \cdot 10^6$	72.7%
<i>BN-x5</i>	$2.1 \cdot 10^6$	73.0%
<i>BN-x30</i>	$2.7 \cdot 10^6$	74.8%
<i>BN-x5-Sigmoid</i>		69.8%



Regularização: *Dropout*

[Srivastava et al., 2014]

“Defina aleatoriamente a saída de alguns neurônios para zero no *forward pass*”
isto é, multiplicar por variáveis aleatórias de Bernoulli com um probabilidade p .

Regularização: *Dropout*

[Srivastava et al., 2014]

“Defina aleatoriamente a saída de alguns neurônios para zero no *forward pass*”
isto é, multiplicar por variáveis aleatórias de Bernoulli com um probabilidade p .

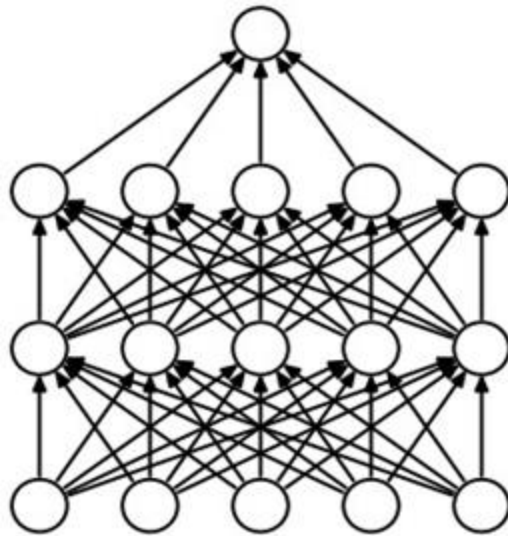
“A ideia principal é descartar aleatoriamente unidades (junto com suas conexões) da rede neural durante o treinamento.”

Regularização: *Dropout*

[Srivastava et al., 2014]

“Defina aleatoriamente a saída de alguns neurônios para zero no *forward pass*”
isto é, multiplicar por variáveis aleatórias de Bernoulli com um probabilidade p .

“A ideia principal é descartar aleatoriamente unidades (junto com suas conexões) da rede neural durante o treinamento.”



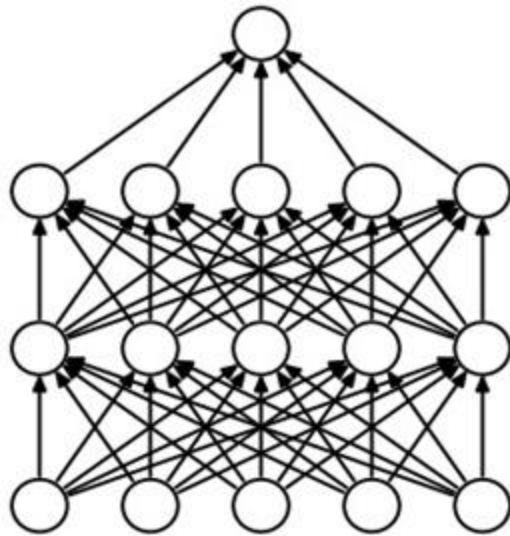
Rede Neural Original

Regularização: *Dropout*

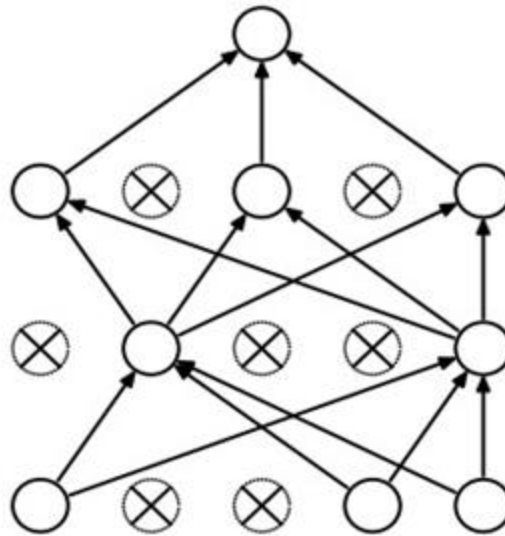
[Srivastava et al., 2014]

“Defina aleatoriamente a saída de alguns neurônios para zero no *forward pass*”
isto é, multiplicar por variáveis aleatórias de Bernoulli com um probabilidade p .

“A ideia principal é descartar aleatoriamente unidades (junto com suas conexões) da rede neural durante o treinamento.”



Rede Neural Original



Rede após *Dropout*

Regularização: *Dropout*

```
p = 0.5 # probability of keeping a unit active. higher = less dropout
```

```
def train_step(X):
```

```
    """ X contains the data """
```

```
    # forward pass for example 3-layer neural network
```

```
    H1 = np.maximum(0, np.dot(W1, X) + b1)
```

```
    U1 = np.random.rand(*H1.shape) < p # first dropout mask
```

```
    H1 *= U1 # drop!
```

```
    H2 = np.maximum(0, np.dot(W2, H1) + b2)
```

```
    U2 = np.random.rand(*H2.shape) < p # second dropout mask
```

```
    H2 *= U2 # drop!
```

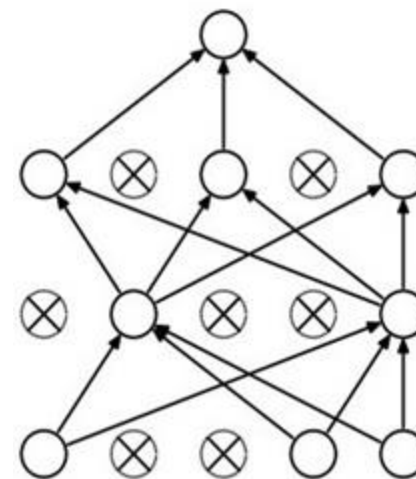
```
    out = np.dot(W3, H2) + b3
```

```
    # backward pass: compute gradients... (not shown)
```

```
    # perform parameter update... (not shown)
```

Exemplo

Forward pass em uma rede de 3 camadas com *dropout*



Regularização: *Dropout*

```
p = 0.5 # probability of keeping a unit active. higher = less dropout
```

```
def train_step(X):
```

```
    """ X contains the data """
```

```
    # forward pass for example 3-layer neural network
```

```
    H1 = np.maximum(0, np.dot(W1, X) + b1)
```

```
    U1 = np.random.rand(*H1.shape) < p # first dropout mask
```

```
    H1 *= U1 # drop!
```

```
    H2 = np.maximum(0, np.dot(W2, H1) + b2)
```

```
    U2 = np.random.rand(*H2.shape) < p # second dropout mask
```

```
    H2 *= U2 # drop!
```

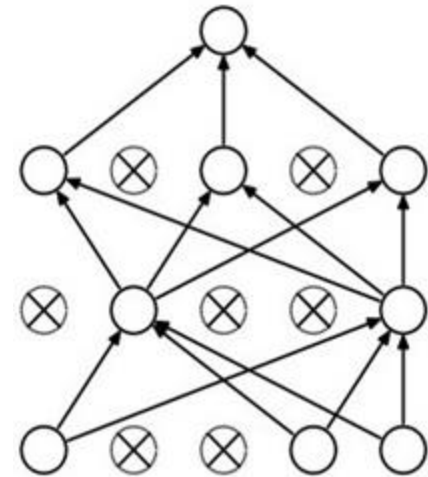
```
    out = np.dot(W3, H2) + b3
```

```
    # backward pass: compute gradients... (not shown)
```

```
    # perform parameter update... (not shown)
```

Exemplo

Forward pass em uma rede de 3 camadas com *dropout*



Regularização: *Dropout*

```
p = 0.5 # probability of keeping a unit active. higher = less dropout
```

```
def train_step(X):
```

```
    """ X contains the data """
```

```
    # forward pass for example 3-layer neural network
```

```
    H1 = np.maximum(0, np.dot(W1, X) + b1)
```

```
    U1 = np.random.rand(*H1.shape) < p # first dropout mask
```

```
    H1 *= U1 # drop!
```

```
    H2 = np.maximum(0, np.dot(W2, H1) + b2)
```

```
    U2 = np.random.rand(*H2.shape) < p # second dropout mask
```

```
    H2 *= U2 # drop!
```

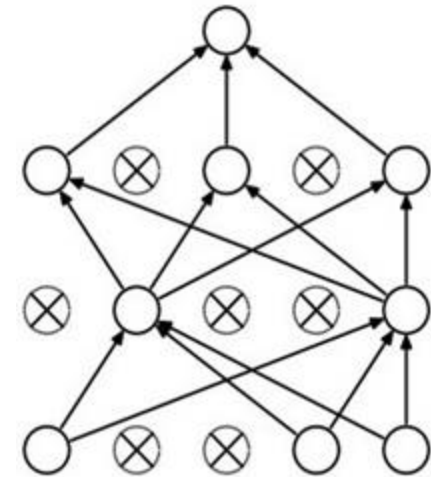
```
    out = np.dot(W3, H2) + b3
```

```
    # backward pass: compute gradients... (not shown)
```

```
    # perform parameter update... (not shown)
```

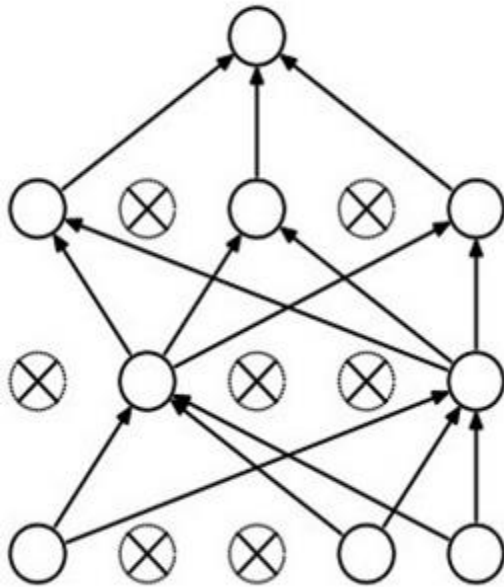
Exemplo

Forward pass em uma rede de 3 camadas com *dropout*



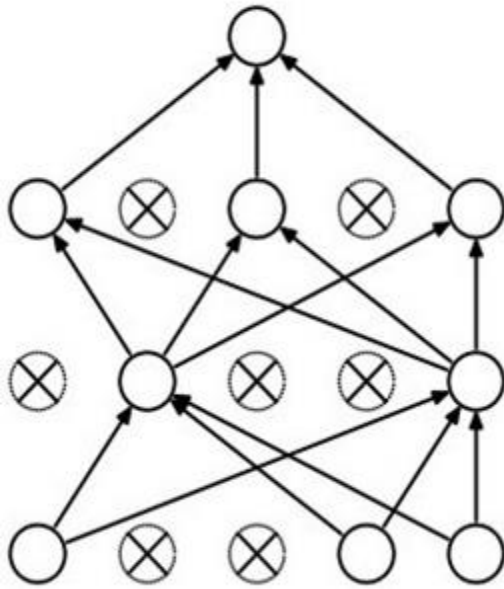
Regularização: *Dropout*

Qual a razão para isso ser uma “boa” ideia?

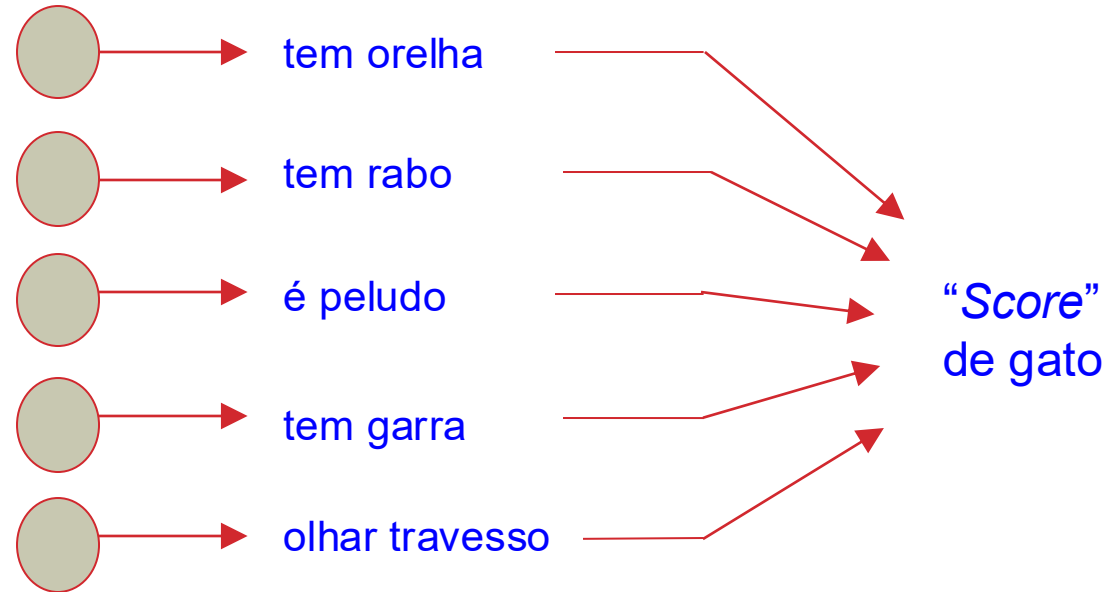


Regularização: *Dropout*

Qual a razão para isso ser uma “boa” ideia?

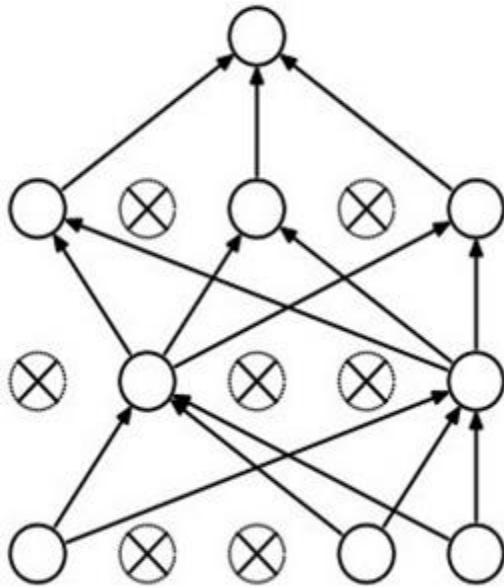


Força a rede a construir uma representação distribuída e redundante



Regularização: *Dropout*

Qual a razão para isso ser uma “boa” ideia?

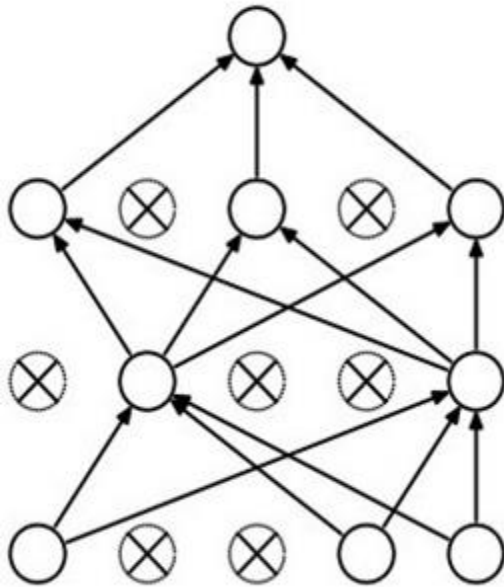


Força a rede a construir uma representação distribuída e redundante



Regularização: *Dropout*

Qual a razão para isso ser uma “boa” ideia?

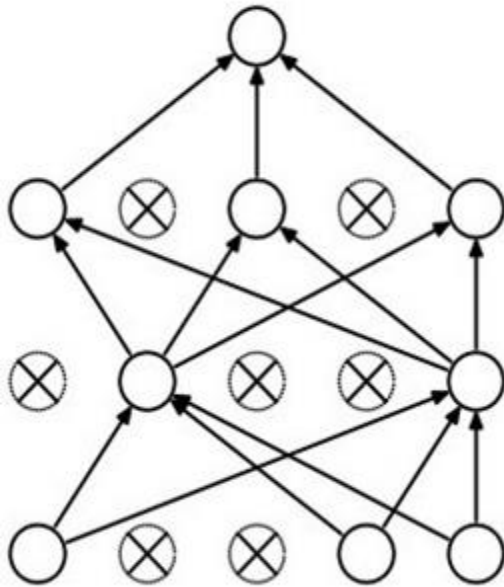


Outra interpretação:

Dropout equivale a treinar uma grande coleção de modelos que compartilham parâmetros – **ensemble**

Regularização: *Dropout*

Qual a razão para isso ser uma “boa” ideia?



Outra interpretação:

Dropout equivale a treinar uma grande coleção de modelos que compartilham parâmetros – **ensemble**

Cada máscara binária corresponde a um modelo, que é treinada em apenas uma parte dos dados (um *minibatch*)

Regularização: *Dropout*

Durante a predição...

Todos os neurônios são mantidos ativos

Regularização: *Dropout*

Durante a predição...

Todos os neurônios são mantidos ativos

⇒ Deve-se ajustar as ativações de forma que para cada neurônio:

Saída na predição = Expectativa de saída no treinamento

Regularização: *Dropout*

Durante a predição...

Todos os neurônios são mantidos ativos

⇒ Deve-se ajustar as ativações de forma que para cada neurônio:

Saída na predição = Expectativa de saída no treinamento → **Inviável !**

Regularização: *Dropout*

Durante a predição...

Todos os neurônios são mantidos ativos

⇒ Deve-se ajustar as ativações de forma que para cada neurônio:

Saída na predição = Expectativa de saída no treinamento → **Inviável !**

```
def predict(X):  
    # ensembled forward pass  
    H1 = np.maximum(0, np.dot(W1, X) + b1) * p # NOTE: scale the activations  
    H2 = np.maximum(0, np.dot(W2, H1) + b2) * p # NOTE: scale the activations  
    out = np.dot(W3, H2) + b3
```

Regularização: *Dropout*

Durante a predição...

Todos os neurônios são mantidos ativos

⇒ Deve-se ajustar as ativações de forma que para cada neurônio:

Saída na predição = Expectativa de saída no treinamento → **Inviável !**

```
def predict(X):  
    # ensembled forward pass  
    H1 = np.maximum(0, np.dot(W1, X) + b1) * p # NOTE: scale the activations  
    H2 = np.maximum(0, np.dot(W2, H1) + b2) * p # NOTE: scale the activations  
    out = np.dot(W3, H2) + b3
```

Solução heurística

Regularização: *Dropout* – Resumo

```
""" Vanilla Dropout: Not recommended implementation (see notes below) """

p = 0.5 # probability of keeping a unit active. higher = less dropout

def train_step(X):
    """ X contains the data """

    # forward pass for example 3-layer neural network
    H1 = np.maximum(0, np.dot(W1, X) + b1)
    U1 = np.random.rand(*H1.shape) < p # first dropout mask
    H1 *= U1 # drop!
    H2 = np.maximum(0, np.dot(W2, H1) + b2)
    U2 = np.random.rand(*H2.shape) < p # second dropout mask
    H2 *= U2 # drop!
    out = np.dot(W3, H2) + b3

    # backward pass: compute gradients... (not shown)
    # perform parameter update... (not shown)

def predict(X):
    # ensembled forward pass
    H1 = np.maximum(0, np.dot(W1, X) + b1) * p # NOTE: scale the activations
    H2 = np.maximum(0, np.dot(W2, H1) + b2) * p # NOTE: scale the activations
    out = np.dot(W3, H2) + b3
```

Regularização: *Dropout* – Resumo

```
""" Vanilla Dropout: Not recommended implementation (see notes below) """

p = 0.5 # probability of keeping a unit active. higher = less dropout

def train_step(X):
    """ X contains the data """

    # forward pass for example 3-layer neural network
    H1 = np.maximum(0, np.dot(W1, X) + b1)
    U1 = np.random.rand(*H1.shape) < p # first dropout mask
    H1 *= U1 # drop!
    H2 = np.maximum(0, np.dot(W2, H1) + b2)
    U2 = np.random.rand(*H2.shape) < p # second dropout mask
    H2 *= U2 # drop!
    out = np.dot(W3, H2) + b3

    # backward pass: compute gradients... (not shown)
    # perform parameter update... (not shown)

def predict(X):
    # ensembled forward pass
    H1 = np.maximum(0, np.dot(W1, X) + b1) * p # NOTE: scale the activations
    H2 = np.maximum(0, np.dot(W2, H1) + b2) * p # NOTE: scale the activations
    out = np.dot(W3, H2) + b3
```

“Drop” no *forward pass*

Regularização: *Dropout* – Resumo

```
""" Vanilla Dropout: Not recommended implementation (see notes below) """

p = 0.5 # probability of keeping a unit active. higher = less dropout

def train_step(X):
    """ X contains the data """

    # forward pass for example 3-layer neural network
    H1 = np.maximum(0, np.dot(W1, X) + b1)
    U1 = np.random.rand(*H1.shape) < p # first dropout mask
    H1 *= U1 # drop!
    H2 = np.maximum(0, np.dot(W2, H1) + b2)
    U2 = np.random.rand(*H2.shape) < p # second dropout mask
    H2 *= U2 # drop!
    out = np.dot(W3, H2) + b3

    # backward pass: compute gradients... (not shown)
    # perform parameter update... (not shown)

def predict(X):
    # ensembled forward pass
    H1 = np.maximum(0, np.dot(W1, X) + b1) * p # NOTE: scale the activations
    H2 = np.maximum(0, np.dot(W2, H1) + b2) * p # NOTE: scale the activations
    out = np.dot(W3, H2) + b3
```

“Drop” no *forward pass*

Ajuste durante a predição

Regularização: *Inverted Dropout*

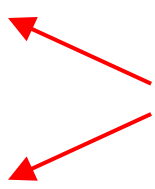
```
p = 0.5 # probability of keeping a unit active. higher = less dropout

def train_step(X):
    # forward pass for example 3-layer neural network
    H1 = np.maximum(0, np.dot(W1, X) + b1)
    U1 = (np.random.rand(*H1.shape) < p) / p # first dropout mask. Notice /p!
    H1 *= U1 # drop!
    H2 = np.maximum(0, np.dot(W2, H1) + b2)
    U2 = (np.random.rand(*H2.shape) < p) / p # second dropout mask. Notice /p!
    H2 *= U2 # drop!
    out = np.dot(W3, H2) + b3

    # backward pass: compute gradients... (not shown)
    # perform parameter update... (not shown)

def predict(X):
    # ensembled forward pass
    H1 = np.maximum(0, np.dot(W1, X) + b1) # no scaling necessary
    H2 = np.maximum(0, np.dot(W2, H1) + b2)
    out = np.dot(W3, H2) + b3
```

Fazer uma divisão por **p**



Regularização: *Inverted Dropout*

```
p = 0.5 # probability of keeping a unit active. higher = less dropout

def train_step(X):
    # forward pass for example 3-layer neural network
    H1 = np.maximum(0, np.dot(W1, X) + b1)
    U1 = (np.random.rand(*H1.shape) < p) / p # first dropout mask. Notice /p!
    H1 *= U1 # drop!
    H2 = np.maximum(0, np.dot(W2, H1) + b2)
    U2 = (np.random.rand(*H2.shape) < p) / p # second dropout mask. Notice /p!
    H2 *= U2 # drop!
    out = np.dot(W3, H2) + b3

    # backward pass: compute gradients... (not shown)
    # perform parameter update... (not shown)

def predict(X):
    # ensembled forward pass
    H1 = np.maximum(0, np.dot(W1, X) + b1) # no scaling necessary
    H2 = np.maximum(0, np.dot(W2, H1) + b2)
    out = np.dot(W3, H2) + b3
```

Fazer uma divisão por **p**

Predição fica inalterada!